

Received 18 February 2025, accepted 21 March 2025, date of publication 26 March 2025, date of current version 7 April 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3554960

## RESEARCH ARTICLE

# LLM-Driven, Self-Improving Framework for Security Test Automation: Leveraging Karate DSL for Augmented API Resilience

EMIL MARIAN PASCA<sup>1</sup>, DANIELA DELINSCHI<sup>1</sup>, RUDOLF ERDEI<sup>1</sup>, AND OLIVIU MATEI<sup>1,2</sup>

<sup>1</sup>Department of Electrical, Electronic and Computer Engineering, Technical University of Cluj Napoca, North University Centre of Baia Mare, Baia Mare 430122, Romania

<sup>2</sup>Department of Research and Development, HOLISUN SRL, Baia Mare 430397, Romania

Corresponding author: Emil Marian Pasca (emil.pasca@campus.utcluj.ro)

This work was supported in part by Romanian National Authority for Scientific Research and Innovation, CCCDI-UEFISCDI, within PNCDI IV under Project ERANET-CHISTERA-IV-TROCI 4/2024; in part by the project “Collaborative Framework for Smart Agriculture (COSA)” through Romania’s National Recovery and Resilience Plan PNRR-III-C9-2022-I8 under Grant 760070; and in part by the CLOUDUT Project, co-funded by European Fund of Regional Development through the Competitiveness Operational Programme 2014–2020, under Contract 235/2020.

**ABSTRACT** Modern software architectures heavily rely on APIs, yet face significant security challenges, particularly with Broken Object Level Authorization (BOLA) vulnerabilities, which remain the most critical API security risk according to OWASP. This paper introduces Karate-BOLA-Guard, an innovative framework leveraging Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) techniques to automate security-focused test case generation for APIs. Our approach integrates vector databases for context retrieval, multiple LLM models for test generation, and observability tools for process monitoring. Initial experiments were carried out on three deliberately vulnerable APIs (VAmPI, Crapi, and OWASP Juice Shop), with subsequent validation on fifteen additional production APIs spanning diverse domains including social media, version control systems, financial services, and transportation services. Our evaluation metrics show Llama 3 8B achieving consistent performance (Accuracy: 3.1-3.4, Interoperability: 3.7-4.3) with an average processing time of 143.76 seconds on GPU. Performance analysis revealed significant GPU acceleration benefits, with 20-25x improvement over CPU processing times. Smaller models demonstrated efficient processing, with Phi-3 Mini averaging 69.58 seconds and Mistral 72.14 seconds, while maintaining acceptable accuracy scores. Token utilization patterns showed Llama 3 8B using an average of 36,591 tokens per session, compared to Mistral’s 25,225 and Phi-3 Mini’s 31,007. Our framework’s effectiveness varied across APIs, with notably strong performance in complex platforms (Instagram: A = 4.3, I = 4.4) while maintaining consistent functionality in simpler implementations (VAmPI: A = 3.6, I = 4.3). The iterative refinement process, evaluated through comprehensive metrics including Accuracy (A), Complexity (C), and Interoperability (I), represents a significant advancement in automated API security testing, offering an efficient, accurate, and adaptable approach to detecting BOLA vulnerabilities across diverse API architectures.

**INDEX TERMS** API security, automation testing tools, cybersecurity, restful API, software testing.

## 1. INTRODUCTION

Application Programming Interfaces (APIs) have become the backbone of modern software architectures, facilitating seamless communication between diverse systems. However,

The associate editor coordinating the review of this manuscript and approving it for publication was Tiago Cruz<sup>1</sup>.

this ubiquity also introduces significant security risks, with BOLA emerging as a critical vulnerability [1]. Traditional security testing methods often struggle to keep pace with the rapid evolution of APIs and the increasing sophistication of cyber threats. As highlighted in [2], traditional security testing methods often struggle to keep pace with the rapid evolution of APIs and the increasing sophistication of cyber

threats, necessitating novel approaches that leverage machine learning and artificial intelligence.

Recent research has demonstrated specific applications of machine learning in API security testing:

- **Anomaly Detection in API Traffic:** Machine learning models, particularly Support Vector Machines (SVM), have successfully identified abnormal API behavior patterns [3]. These methods analyze features such as bandwidth usage and request frequency to detect potential security breaches. However, the study exclusively employs SVM with a linear kernel, which may not capture complex, non-linear attack patterns, and it does not address adversarial robustness—i.e., the ability to resist evasion techniques where attackers subtly modify API requests.
- **Traffic Classification:** Deep learning techniques have proven effective in predicting the validity of API requests [4], offering a proactive approach to identifying vulnerabilities before exploitation. Nonetheless, the study focuses primarily on detecting data breaches and DoS attacks, while neglecting other critical attack vectors such as API abuse, business logic flaws, injection attacks, and broken authentication vulnerabilities.
- **Pattern Recognition:** Neural networks have achieved impressive results in vulnerability detection, with accuracies reaching 94.89% in identifying security flaws through program slice analysis [5]. However, the research is limited to C/C++ programs, which constrains its applicability to software developed in other programming languages.

Despite these advancements, in previous work [2] we identified several critical limitations in current API security testing approaches:

- Current tools primarily focus on functional testing, with limited capability for security-specific test generation;
- Existing automated solutions lack systematic approaches for identifying complex vulnerabilities like BOLA;
- Most tools operate in a “brute-force” testing mode without understanding inter-parameter dependencies, leading to high numbers of invalid requests;
- The majority of security flaws discovered by existing tools are found by chance rather than through targeted testing.

Our framework addresses these specific limitations through:

- A modular architecture that allows for easy integration of different LLMs and testing tools;
- RAG-enhanced test generation for improved context-awareness and relevance;
- An iterative, self-improving approach to test case generation and refinement;
- Integrated observability tools for continuous monitoring and improvement of test quality.

Our research is guided by the following questions:

- **RQ 1** - To what extent does the integration of RAG techniques enhance the quality and relevance of

generated Karate DSL test cases for API security testing, particularly focusing on BOLA vulnerabilities?

- **RQ 2** - How does our RAG-enhanced approach impact the coverage and specificity of generated test cases for API endpoints potentially vulnerable to BOLA?
- **RQ 3** - What are the trade-offs between test generation time, accuracy, and comprehensiveness when employing RAG techniques in API security testing?

Our approach directly addresses the gaps identified in previous systematic reviews of API security testing methodologies. While Barabanov et al. [6] advanced systematic approaches for BOLA vulnerability detection using OpenAPI specifications, and Huang et al. [7] provided insights through analysis of real-world BOLA vulnerabilities, neither fully addressed the automation challenges in security testing. By combining LLMs with RAG techniques, our framework specifically targets the limitations identified in [2], offering:

- Targeted security testing that goes beyond random exploration;
- Continuous learning and refinement of test cases based on execution results;
- Systematic approaches for identifying and testing potential BOLA vulnerabilities;

This systematic approach represents a significant advancement in automated API security testing, moving beyond the current limitations of chance-based vulnerability discovery toward targeted, intelligent test generation.

## A. PAPER STRUCTURE

This paper is structured as follows: Section II by providing essential background on BOLA vulnerabilities and reviewing related work in API security testing and LLM applications. Section III details our methodology, including a comprehensive description of the Karate-BOLA-Guard architecture, our experimental setup, followed by a thorough analysis of our results in Section IV, which offers a discussion of our findings, addressing both the implications and limitations of our approach. Finally, Section V and VI, concludes the paper with a summary of our key contributions and directions for future research.

## II. BACKGROUND AND RELATED WORK

This section provides an overview of the current landscape in API security testing, focusing on the critical nature of BOLA vulnerabilities, traditional and emerging approaches to API testing, and the growing role of LLMs in software testing. It also introduces RAG techniques as a promising avenue for enhancing LLM capabilities in API security testing.

### A. BOLA VULNERABILITIES IN API SECURITY

Broken Object Level Authorization (BOLA) is recognized as one of the most critical vulnerabilities in API security. OWASP consistently ranks BOLA as the most severe API security risk due to its prevalence and potential for exploitation [1]. This vulnerability arises when APIs fail to

enforce proper authorization checks, allowing unauthorized access or manipulation of resources.

The consequences of BOLA vulnerabilities include unauthorized data access, privilege escalation, and potential data manipulation [8]. Addressing BOLA vulnerabilities is particularly challenging due to the complexity of modern API architectures and the difficulty of implementing consistent authorization mechanisms.

Emerging AI-driven approaches have shown promise in tackling these challenges. For instance, *Purple Llama CyberSecEval* [9] introduces a secure coding benchmark that aids in detecting vulnerabilities, including BOLA, within language model-generated code. Additionally, the SALLM framework proposed by Siddiq et al. [10] benchmarks the ability of LLMs to generate secure code, addressing common vulnerabilities like BOLA. These approaches demonstrate the potential of AI to enhance the detection and mitigation of BOLA vulnerabilities.

### B. CURRENT APPROACHES TO API SECURITY TESTING

Traditional API security testing methods, such as manual penetration testing and automated scanning tools, face limitations in addressing the evolving complexity of modern APIs and advanced cyber threats [11]. These methods often lack the scalability and adaptability required to identify context-specific vulnerabilities like BOLA.

Recent advancements in API security testing aim to overcome these limitations through innovative approaches, including:

- **Specification-Based Testing:** Generating test cases based on API documentation to ensure adherence to expected behavior [12];
- **Machine Learning Techniques:** Leveraging anomaly detection algorithms to identify potential vulnerabilities [8];
- **Automated Test Case Generation:** Frameworks like RESTest employ diverse strategies to create comprehensive test suites that improve coverage and accuracy [11];
- **Adaptive Test Generation with LLMs:** Approaches like *TestPilot*, introduced by Schäfer et al. [13], utilize LLMs to generate adaptive test cases, improving test coverage and uncovering unique API vulnerabilities.
- **Automated testing of REST APIs** has seen significant advancements, with tools like AutoRestTest [14] adopting innovative techniques such as Multi-Agent Reinforcement Learning (MARL) and Semantic Property Dependency Graphs (SPDG) for dependency-aware test generation. Our work, unlike AutoRestTest, which focuses on general fault detection, our framework prioritizes security-specific weaknesses through a RAG pipeline.

This distinction highlights the complementary nature of our work. AutoRestTest demonstrates high generalization for REST API fault detection, while our approach narrows its focus to BOLA vulnerabilities, employing

iterative syntax validation and domain-specific test optimization.

While these advancements have enhanced the efficiency and effectiveness of API security testing, they still fall short in addressing the nuanced and context-specific nature of vulnerabilities like BOLA. There remains a pressing need for methodologies that integrate adaptive, AI-driven frameworks capable of providing comprehensive, scalable, and precise security testing solutions.

### C. LLMs IN SOFTWARE TESTING

The application of LLMs in software testing has gained significant traction in recent years. LLMs have demonstrated effectiveness in various testing tasks, including test case preparation, and automated test generation [15].

Key developments in this area include:

- The use of LLMs for generating unit tests, potentially streamlining the software testing process [16];
- Application of LLMs in fuzzing techniques, addressing the need for more automated and evolving vulnerability detection methods [17];
- Exploration of LLMs as AI sparring partners for both high-level task planning and low-level vulnerability hunting in penetration testing scenarios [18];
- Generating security-centric assertions for hardware and software systems, such as those outlined by Kande et al. in *Security Assertions by Large Language Models* [19].

However, the use of LLMs in software testing is not without challenges. Researchers have cautioned that LLMs should be used carefully, as guidelines for their secure adoption are still being developed [20].

### D. RAG TECHNIQUES IN AI-ASSISTED TESTING

Retrieval-Augmented Generation (RAG) represents a promising approach to enhance the capabilities of LLMs in specific domains. By combining the generative power of LLMs with the ability to retrieve and incorporate relevant information from a curated knowledge base, RAG techniques have the potential to produce more accurate and context-aware outputs [21].

In the context of API security testing, RAG techniques offer several potential benefits:

- Improved relevance and specificity of generated test cases;
- Enhanced ability to incorporate domain-specific knowledge and best practices;
- Potential for generating more diverse and comprehensive test scenarios, as demonstrated by *Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing* [22].

Despite these promising aspects, applying RAG techniques in API security testing, particularly for BOLA vulnerabilities, remains largely unexplored. Recent advancements, such as the findings in *LLMSecEval: Evaluating the Security of LLM-Generated Code* [23], provide a foundational basis

for leveraging LLM capabilities in test generation. Our work, however, aims to address this gap by developing and evaluating a RAG-enhanced framework specifically designed for BOLA vulnerability detection and mitigation.

### E. FRAMEWORK TEST GENERATION PERFORMANCE EVALUATION

While test coverage is a widely used metric in functional testing, indicating the extent to which a service's functionality is covered by tests, it is not an appropriate measure for our framework. Our focus on non-functional testing, specifically oriented towards BOLA vulnerability discovery, necessitates a different approach to performance evaluation.

In light of this, we have decided to employ the metrics defined in our previous work [24], which are more suitable for assessing the performance of our framework in generating security-focused test cases. These metrics provide a more relevant and accurate representation of our framework's effectiveness in the context of BOLA vulnerability detection.

To ensure clarity and facilitate a comprehensive understanding of our evaluation process, we will revisit these metrics in the following subsection. This recapitulation is crucial as these metrics form the basis for the scoring system we have applied to the generated tests, allowing for a nuanced assessment of our framework's performance in generating BOLA-oriented test cases.

### F. EVALUATION METRICS

Standard test evaluation metrics, such as path or branch coverage and functional test coverage, are inadequate for accurately assessing the performance of our proposed framework. Although cybersecurity evaluation suites for LLMs have been introduced in prior research [18], these frameworks primarily focus on evaluating the capabilities of LLMs themselves in producing cybersecurity oriented code, rather than the frameworks or the quality of the test cases they generate. To address this gap, we have defined three bespoke metrics to facilitate a rigorous and objective evaluation of the tests produced by our framework. We proposed simplified metrics framework that offers several advantages over the original weighted approach that we used in past experiments [24] while requiring some adjustments to better align with BOLA vulnerability detection and the constraints of non-execution testing, which are required in our case, since we don't execute our test against production API, their focus is on generation.

#### 1) ACCURACY

The accuracy metric requires the most significant revision to account for analysis of BOLA vulnerabilities:

$$A = 1 + \sum_{i=1}^4 a_i \quad (1)$$

where:

- $a_1$ : **BOLA Pattern Coverage** — Coverage of known BOLA vulnerability patterns (0-1);
- $a_2$ : **Authorization Boundary Testing** — Thoroughness in testing authorization boundaries (0-1);
- $a_3$ : **Edge Case Handling** — Inclusion of edge cases and corner scenarios (0-1);
- $a_4$ : **Validation Completeness** — Completeness of response validation logic (0-1).

#### 2) COMPLEXITY

The complexity metric is reframed to emphasize BOLA testing scenarios:

$$C = 1 + \sum_{i=1}^4 c_i \quad (2)$$

where:

- $c_1$ : **Scenario Structure** — Clarity and organization of the test scenario (0-1);
- $c_2$ : **Authorization Logic** — Sophistication of authorization checks and validations (0-1);
- $c_3$ : **Data Manipulation** — Complexity of data handling and state management (0-1);
- $c_4$ : **Assertion Coverage** — Comprehensiveness of validation checks (0-1).

#### 3) INTEROPERABILITY

The current definition should be modified to focus more on Karate DSL compatibility and BOLA testing requirements:

$$I = 1 + \sum_{i=1}^4 i_i \quad (3)$$

where:

- $i_1$ : **Syntax Correctness** — Adherence to Karate DSL syntax and structure (0-1);
- $i_2$ : **Authentication Integration** — Proper handling of authentication mechanisms and tokens (0-1);
- $i_3$ : **Framework Compatibility** — Alignment with Karate DSL best practices and features (0-1);
- $i_4$ : **Data Handling** — Appropriate management of test data and variables (0-1).

This form of evaluation emphasizes the aspects of complexity specific to BOLA vulnerability testing while maintaining measurability without execution.

#### 4) OTHER ADVANTAGES OF THIS EVALUATION FORM

- 1) **Static Evaluation:** All metrics can be assessed through static analysis of the generated test code, eliminating the need for execution;
- 2) **BOLA Focus:** The metrics specifically target characteristics relevant to BOLA vulnerability detection;
- 3) **Karate DSL Alignment:** Metrics are tailored to evaluate tests written in Karate DSL format but could probably be extended to other framework as well.



## 5) EVALUATION IMPLEMENTATION GUIDELINES

*a: ACCURACY (A) SCORING GUIDELINES***BOLA Pattern Coverage ( $a_1$ )**

- 1.0: Covers all relevant BOLA patterns for the endpoint;
- 0.7: Covers most common BOLA patterns;
- 0.3: Basic BOLA checks only;
- 0.0: Missing critical BOLA patterns.

**Authorization Boundary Testing ( $a_2$ )**

- 1.0: Complete coverage of authorization boundaries;
- 0.7: Good boundary testing with minor gaps;
- 0.3: Basic boundary testing only;
- 0.0: Missing crucial boundary tests.

**Edge Case Handling ( $a_3$ )**

- 1.0: Comprehensive edge case coverage;
- 0.7: Most important edge cases covered;
- 0.3: Basic edge cases only;
- 0.0: Missing critical edge cases.

**Validation Completeness ( $a_4$ )**

- 1.0: Complete validation of all responses;
- 0.7: Good validation coverage with minor gaps;
- 0.3: Basic validation only;
- 0.0: Missing critical validations.

*b: COMPLEXITY (C) SCORING GUIDELINES***Scenario Structure ( $c_1$ )**

- 1.0: Clear, logical flow with excellent organization;
- 0.7: Well-structured with minor organizational issues;
- 0.3: Basic structure with some unclear flow;
- 0.0: Poor organization or illogical flow.

**Authorization Logic ( $c_2$ )**

- 1.0: Comprehensive authorization checks with edge cases;
- 0.7: Good coverage of authorization scenarios;
- 0.3: Basic authorization checks only;
- 0.0: Inadequate or missing authorization checks.

**Data Manipulation ( $c_3$ )**

- 1.0: Sophisticated data handling with proper state management;
- 0.7: Good data manipulation with minor state issues;
- 0.3: Basic data manipulation only;
- 0.0: Poor or incorrect data manipulation.

**Assertion Coverage ( $c_4$ )**

- 1.0: Comprehensive assertions covering all aspects;
- 0.7: Good assertion coverage with minor gaps;
- 0.3: Basic assertions only;
- 0.0: Missing or incorrect assertions.

*c: INTEROPERABILITY (I) SCORING GUIDELINES***Syntax Correctness ( $i_1$ )**

- 1.0: Perfect Karate DSL syntax with no issues;
- 0.7: Very minor syntax issues that don't affect functionality;
- 0.3: Multiple minor syntax issues;

**Feature:** User Profile Access Control Testing

**Scenario:** Verify user cannot access other user's profile

**Given** path 'api/users'

**And** path otherId

**And** header Authorization = userToken

**When** method get

**Then** status 403

**And** match response contains { error: '#string' }

**Listing 1. Sample test for scoring example.**

- 0.0: Major syntax errors that prevent execution.

**Authentication Integration ( $i_2$ )**

- 1.0: Complete and correct authentication handling with proper token management;
- 0.7: Proper authentication but minor token handling issues;
- 0.3: Basic authentication without proper token management;
- 0.0: Missing or incorrect authentication handling.

**Framework Compatibility ( $i_3$ )**

- 1.0: Full utilization of Karate DSL features and best practices;
- 0.7: Good use of framework features with minor deviations;
- 0.3: Basic framework usage with some anti-patterns;
- 0.0: Poor framework utilization or misuse of features.

**Data Handling ( $i_4$ )**

- 1.0: Excellent data management with proper variable scoping;
- 0.7: Good data handling with minor inefficiencies;
- 0.3: Basic data management with some issues;
- 0.0: Poor data handling or incorrect variable usage.

**G. SAMPLE SCORING EXAMPLE**

For a generated test case:

## 1) ACCURACY (A)

- **BOLA Pattern Coverage ( $f_1$ ):** This test seems to target a common BOLA pattern.  $f_1 = 0.7$
- **Authorization Boundary Testing ( $f_2$ ):** The test checks a boundary.  $f_2 = 0.7$
- **Edge Case Handling ( $f_3$ ):** The test doesn't seem to include specific edge cases.  $f_3 = 0.3$
- **Validation Completeness ( $f_4$ ):** The test validates the status code and part of the error message.  $f_4 = 0.7$

**Total Accuracy Score:**  $A = 1 + (0.5 + 0.7 + 0.3 + 0.5) = 3$

While the test case demonstrates coverage of a fundamental BOLA vulnerability pattern and incorporates negative testing scenarios, it lacks specificity in several critical areas. The test does not explicitly define values for user identification, authentication tokens, or detailed assertions. Nevertheless, the expected response validation—incorporating both an error status code and corresponding error message—aligns with established API security testing

practices. The implementation of status code verification (403) and error message validation provides a baseline for authorization boundary testing, though more comprehensive assertions would strengthen the test's effectiveness."

## 2) COMPLEXITY (C)

- **Scenario Structure** ( $s_1$ ): The scenario is simple and focused.  $c_1 = 0.7$
- **Authorization Logic** ( $s_2$ ): The test checks authorization.  $c_2 = 0.7$
- **Data Manipulation** ( $s_3$ ): The test uses a variable `otherId`.  $c_3 = 0.7$
- **Assertion Coverage** ( $s_4$ ): The test checks the status code and error message.  $c_4 = 0.7$

**Total Complexity Score:**  $C = 1 + (1.0 + 0.5 + 0.7 + 0.8) = 4$  The GET request operations demonstrate relatively moderate complexity levels. While the implementation includes Authorization headers for token-based authentication, their validation mechanisms may not provide comprehensive security coverage. Furthermore, these test scenarios necessitate supplementary utility functions to manage token generation processes and user identification handling, increasing the overall implementation complexity and maintenance requirements.

## 3) INTEROPERABILITY (I)

- **Syntax Correctness** ( $i_1$ ): The Karate syntax looks good.  $i_1 = 1.0$ ;
- **Authentication Integration** ( $i_2$ ): The test uses an Authorization header.  $i_2 = 1$  (Assuming `userToken` is handled correctly);
- **Framework Compatibility** ( $i_3$ ): The test uses standard Karate features.  $i_3 = 1.0$ ;
- **Data Handling** ( $i_4$ ): The test uses a variable `otherId` and `userToken`.  $i_4 = 0.5$  (Needs manual intervention to setup the variables to the appropriate values.);

**Total Interoperability Score:**  $I = 1 + (1.0 + 0.8 + 1.0 + 0.5) = 4.3$ .

From an interoperability perspective, the test case achieves a score of 4.3 out of 5.0, demonstrating strong framework integration while revealing specific areas for enhancement. The implementation exhibits exemplary syntax adherence ( $i_1 = 1.0$ ) and leverages Karate DSL's native features effectively ( $i_3 = 1.0$ ). The authentication mechanism is appropriately integrated through the standard Authorization header implementation ( $i_2 = 0.8$ ). However, the data handling component ( $i_4 = 0.5$ ) presents a notable limitation due to its reliance on undefined variables (`otherId` and `userToken`), which necessitates manual configuration prior to test execution.

While the overall interoperability score indicates strong framework integration, addressing the data handling limitations would further enhance the test's portability and reduce the need for manual intervention in test setup procedures.

## 4) NOTES

- 1) We had used measurement values from 0.1 - 1.0, having in mind the thresholds mentioned in the Section II-F5.
- 2) These scores would depend also on more information about the API and the test environment.
- 3) The scoring guidelines are still subjective in a certain amount, as different evaluators might give slightly different scores due to subjective measurements like (excellent, good, poor, basic).
- 4) This evaluation method focuses on static analysis of the test code. It doesn't tell us if the test actually finds any vulnerabilities.

## H. LIMITATIONS AND BROADER QUALITY CONSIDERATIONS

### 1) GENERAL CODE QUALITY METRICS NOT INCLUDED

While important for overall code quality, several traditional software metrics have been intentionally excluded from our framework:

#### 1) Code Maintainability

- Readability and documentation quality;
- Code structure and organization;
- Comment completeness and clarity.

These aspects, while valuable, are general software engineering concerns that apply to all code rather than being specific to security test generation.

#### 2) Cognitive Complexity

- Intellectual effort required to understand the test;
- Learning curve for new team members;
- Mental model complexity.

These factors are subjective and may vary based on individual expertise and background.

#### 3) Dependencies and Environmental Factors

- External library dependencies;
- Environment setup requirements;
- Cross-platform compatibility.

These considerations are more relevant to long-term maintenance than to the immediate quality of generated security tests.

## I. RATIONALE FOR THE UNIFIED APPROACH

- 1) **Focus on Generation:** Each factor has been chosen to reflect critical aspects of security testing. For example, Interoperability emphasizes secure integration; Complexity focuses on maintainability and operational simplicity to mitigate risks of configuration errors; and Accuracy directly addresses the test's ability to uncover vulnerabilities.
- 2) **Objectivity:** The chosen metrics emphasize objectively measurable aspects of test quality specific to BOLA vulnerability detection.
- 3) **Automation Compatibility:** The selected metrics can be evaluated programmatically, making them suitable for automated assessment of generated tests.

- 4) **Applicability to Generated Tests:** In frameworks such as Karate DSL, tests are often structured in a Given–When–Then format. This inherent structure facilitates the extraction and quantification of the proposed factors (e.g., the number of steps, the intricacy of request bodies, and assertion logic), making the framework both practical and relevant for automated security assessments.

#### J. FUTURE CONSIDERATIONS

- 1) **Automation Potential:** The simplified scoring system facilitates automated evaluation through static analysis tools.
- 2) **Extensibility:** The framework can be extended to cover other types of API vulnerabilities while maintaining the same structure.
- 3) **Validation:** Consider developing a benchmark suite of known BOLA vulnerabilities to validate the scoring system.
- 4) **Complementary Metrics:** Future work could explore how to integrate traditional code quality metrics as supplementary measures without compromising the framework’s focus on security test generation.

#### K. OVERVIEW OF KARATE-BOLA-GUARD FRAMEWORK

Karate-BOLA-Guard is designed as a modular, self-improving framework that leverages LLMs and RAG techniques to automate the creation and refinement of API security tests. The framework specifically targets BOLA vulnerabilities, generating test cases in Karate DSL format.

#### L. FRAMEWORK ARCHITECTURE

Figure 1 illustrates our framework’s architecture, where yellow components represent data storage and processing elements, while the blue-highlighted Karate Test Runner operates in an isolated container. The framework processes OpenAPI Specifications through several key stages: The OAS Parser breaks down API specifications into manageable fragments, storing them alongside BOLA and Karate DSL examples in the Vector Data Storage. The LLM Service interacts with our RAG System to generate contextually-aware test cases, which undergo syntax verification before execution. The Karate Test Runner executes these tests against containerized target applications, with results feeding into our analysis pipeline. The Test Result Analyzer processes execution outcomes, generating reports that feed back into the Vector Data Storage for continuous improvement. The LLM Observability Service monitors the entire process, tracking performance metrics and system behavior. This cyclic flow ensures continuous refinement of test case generation and execution, while maintaining component isolation for security and scalability. Each component serves a specific purpose in our security testing pipeline:

- **OAS Parser:** Decomposes API endpoint information into manageable fragments.

- **Vector Database:** Stores API fragments and historical test data for efficient retrieval.
- **RAG System:** Queries the vector database for BOLA-related vulnerability tests to augment the LLM’s context for test generation.
- **LLM:** Generates initial Karate test scripts based on the augmented context.
- **Syntax and Compatibility Verifier:** Ensures generated tests are syntactically correct and compatible with the Karate framework.
- **Karate Test Runner:** Executes generated tests against the target API in a containerized environment.
- **Error Analysis:** Analyzes test results and failures to inform future test generation.
- **Observability Tool:** Monitors and logs the entire process for analysis and improvement.

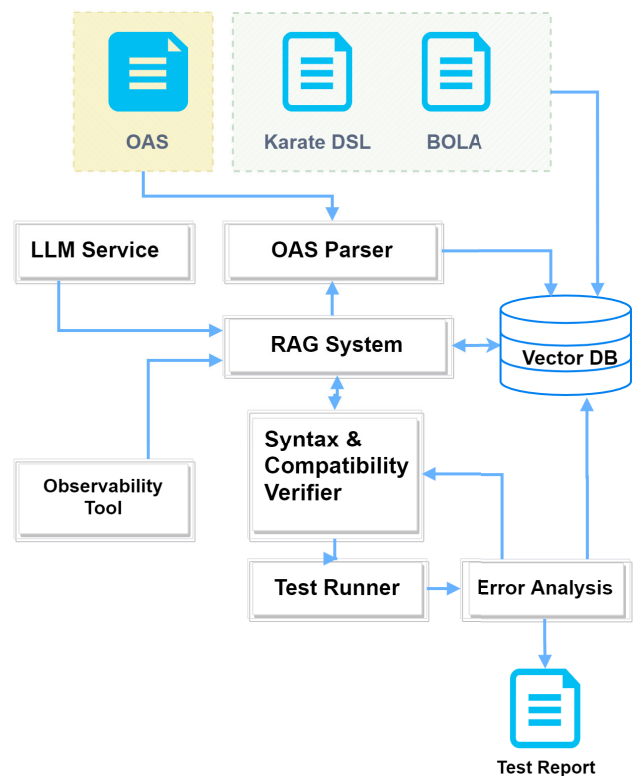


FIGURE 1. Karate-bola-guard framework architecture.

#### M. RAG-ENHANCED TEST GENERATION PROCESS

Our RAG-enhanced test generation process leverages these components in a sequential workflow:

- 1) **Fragment Retrieval:** The RAG system queries the vector database for relevant API fragments based on the current testing context.
- 2) **Prompt Augmentation:** Retrieved fragments are used to enhance the prompt sent to the LLM.
- 3) **Test Scenario Generation:** The LLM generates test scenarios based on the augmented prompt.

- 4) **Karate DSL Translation:** The generated scenarios are translated into valid Karate DSL syntax.
- 5) **Compatibility Verification:** The Syntax and Compatibility Verifier ensures the correctness of the generated tests.
- 6) **Test Execution:** The Karate Test Runner executes the verified tests against the target API.
- 7) **Result Analysis:** The Error Analysis component examines test results, feeding insights back into the system for future test generation.

This integrated process ensures that generated tests are both relevant to the specific API under test and focused on potential BOLA vulnerabilities. By combining RAG techniques with LLM-driven test generation and rigorous verification, our framework aims to provide comprehensive and accurate API security testing, particularly for detecting BOLA vulnerabilities.

#### N. KARATE-BOLA-GUARD FRAMEWORK ALGORITHM

The following algorithm represents the Karate-BOLA-Guard framework, where the main loop iterates over each retrieved fragment, and the inner loop handles the error-checking and refinement process for each generated test case. The number of iterations is determined by the number of retrieved fragments ( $k$ ).

---

##### Algorithm 1 Karate-BOLA-Guard Framework

---

```

Initialize  $\mathcal{V} \leftarrow \text{Fragment}(\mathcal{A})$ 
 $\mathcal{F}' \leftarrow \text{Retrieve}(\mathcal{V}, q, k)$ 
foreach  $f \in \mathcal{F}'$  do
   $\mathcal{T} \leftarrow \text{Generate}(\mathcal{L}, f, c)$ 
   $\mathcal{T}' \leftarrow \text{Verify}(\mathcal{T})$ 
   $\mathcal{R} \leftarrow \text{Execute}(\mathcal{T}')$ 
   $\mathcal{E} \leftarrow \text{Analyze}(\mathcal{R})$ 
   $\mathcal{V} \leftarrow \text{Update}(\mathcal{V}, \mathcal{E})$ 
  for  $j = 1$  to  $n$  do
    if  $\text{HasErrors}(\mathcal{R})$  then
       $\mathcal{T}' \leftarrow \text{Verify}(\mathcal{T}', \mathcal{E})$ 
       $\mathcal{R} \leftarrow \text{Execute}(\mathcal{T}')$ 
       $\mathcal{E} \leftarrow \text{Analyze}(\mathcal{R})$ 
       $\mathcal{V} \leftarrow \text{Update}(\mathcal{V}, \mathcal{E})$ 
    else
      break
return Final set of refined test cases  $\mathcal{T}'$ 

```

---

#### Legend and Description of Functions:

itemsep=0pt, parsep=0pt, topsep=0pt

- $\mathcal{A}$ : Set of API endpoints;
- $\mathcal{V}$ : Vector database;
- $\mathcal{F}'$ : Set of retrieved API fragments;
- $\mathcal{L}$ : Language model;
- $\mathcal{T}$ : Set of generated test cases;
- $\mathcal{T}'$ : Set of verified test cases;
- $\mathcal{R}$ : Set of test results;

- $\mathcal{E}$ : Set of error information;
- $q$ : Query for fragment retrieval;
- $k$ : Number of fragments to retrieve;
- $c$ : Context for test generation;
- $n$ : Maximum number of error-checking iterations.

#### Functions:

- 1) **Fragment** :  $\mathcal{A} \rightarrow \mathcal{V}$   
Decomposes API endpoints into fragments and stores them in the vector database;
- 2) **Retrieve** :  $\mathcal{V} \times \mathcal{Q} \times \mathbb{N} \rightarrow \mathcal{F}'$   
Retrieves  $k$  most relevant fragments from  $\mathcal{V}$  based on query  $q \in \mathcal{Q}$ ;
- 3) **Generate** :  $\mathcal{L} \times \mathcal{F}' \times \mathcal{C} \rightarrow \mathcal{T}$   
Generates test cases using language model  $\mathcal{L}$ , fragment  $f \in \mathcal{F}'$ , and context  $c \in \mathcal{C}$ ;
- 4) **Verify** :  $\mathcal{T} \times \mathcal{E} \rightarrow \mathcal{T}'$   
Verifies and corrects syntax of generated test cases, optionally using error information;
- 5) **Execute** :  $\mathcal{T}' \rightarrow \mathcal{R}$   
Executes test cases and produces results;
- 6) **Analyze** :  $\mathcal{R} \rightarrow \mathcal{E}$   
Analyzes test results and extracts error information;
- 7) **Update** :  $\mathcal{V} \times \mathcal{E} \rightarrow \mathcal{V}$   
Updates vector database with new error information;
- 8) **HasErrors** :  $\mathcal{R} \rightarrow \{\text{true}, \text{false}\}$   
Checks if the execution results contain errors.

#### 1) ITERATIVE REFINEMENT PROCESS

Karate-BOLA-Guard employs an iterative approach to test refinement, continuously improving the quality and relevance of generated tests, these can be improved drastically, slightly, or sometimes, less, in the Listing 2 example, we can see the initial generated test, which contains small artifacts like single quote or backslash escape characters, or even spacing that can affect the test compilation, and after one or several iterations we get a result as seen in Listing 3, where we have the correct formatting, as well, different characters removed.

**Feature:** Testing for Broken Authentication / Authorization Vulnerabilities

```

Scenario: Check for missing authentication requirement
Given url http://192.168.1.106/users/v1/register
And request { username: 'existingUser ',
  password: 'Password123!' }
When method post
Then status 401
And match response contains \'
  Authentication required\'.

```

**Listing 2. Code Snippet of initial test generated.**

The test iteration consists of several calls to LLM service, Vector DB, and reusing refinement functions to improve the generated test, however, the lifecycle as illustrated in Fig. 2, can be described in the following key stages:



**Feature:** Testing for Broken Authentication/Authorization Vulnerabilities

**Scenario:** Check for missing authentication requirement  
**Given** url "http://192.168.1.106/users/v1/register"  
**And request** {  
    username: "existingUser"  
    password: "Password123!"  
}  
**When method** post  
**Then status** 401  
**And match response contains** "Authentication required".

**Listing 3.** Code Snippet of final version of test generated.

- 1) Initial test is generated by the “RAG Test Generation Components” system;<sup>1</sup>
- 2) The test enters the analysis stage, where it undergoes analysis and initial refinement;
- 3) The refinement process identifies and removes errors, producing an improved version ready for execution;
- 4) The refined test is passed to the “Test Runner” for execution. If interoperability issues arise (such as compilation errors), the test is redirected back to the analysis components;
- 5) A final refinement cycle produces the third iteration of the test. This cycle continues until either successful execution is achieved (resulting in a test report) or the maximum iteration limit is reached.

Several important aspects characterize this iterative process:

- Every generated test undergoes at least one refinement iteration for error checking;
- The process is bounded by a maximum number of iterations as defined in Algorithm 1;
- Fig. 6 demonstrates the improvement in evaluation metrics (accuracy, complexity, and interoperability) between initial and final iterations across different APIs and models.

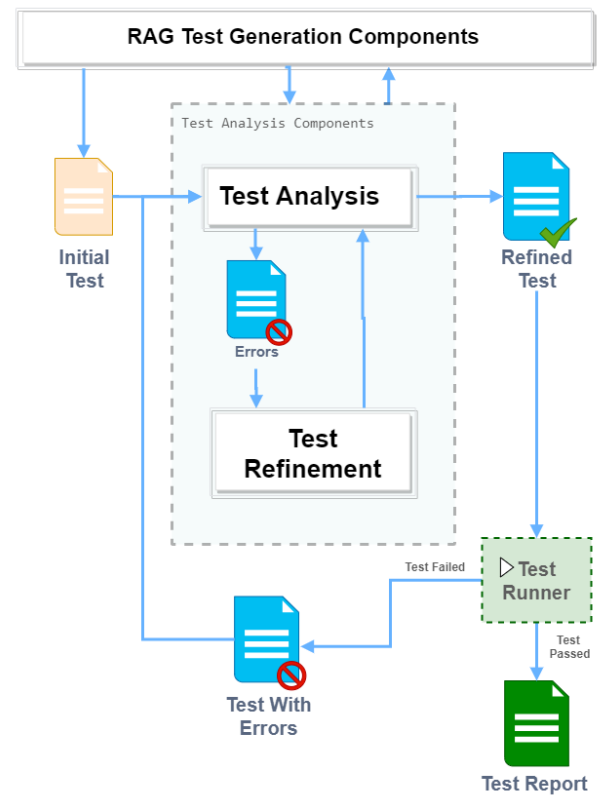
### III. METHODOLOGY

This section describes the research methodology employed in developing and evaluating the Karate-BOLA-Guard framework. We detail our research design, data collection methods, experimental procedures, and data analysis techniques.

#### A. RESEARCH DESIGN

Our study employed an experimental research design to evaluate the effectiveness of the Karate-BOLA-Guard framework in writing test cases towards BOLA vulnerabilities detection in APIs. We used a combination of quantitative and qualitative methods to assess the performance of different LLMs within our framework.

<sup>1</sup>The internal complexity of this system is abstracted for clarity in this flow diagram.



**FIGURE 2.** Test iteration flowchart diagram.

#### B. EXPERIMENTAL SETUP

The experimental design for this research was carefully constructed to balance accessibility and reproducibility with the need for controlled testing conditions. Our approach incorporated mid-range hardware and open-source software to facilitate replication and further experimentation by other researchers and practitioners in the field.

We deliberately selected a mid-range hardware configuration and open-source software suite to ensure that our experimental setup could be easily replicated. This decision was made to encourage further research and validation of our findings by the broader academic and professional community. The specific details of our hardware and software environment are provided in the following subsection.

Given the potential unpredictability of outcomes and the possible impact on tested systems, we made a strategic decision to conduct our initial research on a single, self-hosted API. This approach allowed us to maintain a controlled environment and mitigate any unforeseen consequences that might arise from testing on diverse, production systems. The criteria for selecting this API and its relevance to our research objectives are detailed in the subsequent section.

This experimental setup was designed to provide a balance between realistic testing conditions and the need for a controlled, reproducible environment. By using a self-hosted API and readily available hardware and software, we aimed to create a framework that could be both rigorously tested and

**TABLE 1. System and software configuration.**

|                          |                                                                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hardware</b>          | RAM: 24 GB<br>CPU: Intel(R) Core(TM) i7-4790K CPU x 8 @ 4.00GHz<br>GPU: NVIDIA GeForce GTX 1070 with 8192MiB                                                                                                          |
| <b>Software Stack</b>    | OS: Ubuntu 24.04 LTS, Python 3.12,<br>Karate DSL 1.4.1, Docker 20.10.12                                                                                                                                               |
| <b>LLM Integration</b>   | GPT4ALL wrapper with:<br>- Mistral Instruct 7B (GGUF Q4 <sub>0</sub> )<br>- Wizard v1.2 13B (GGUF Q4 <sub>0</sub> )<br>- Phi-3 Mini Instruct (GGUF Q4 <sub>0</sub> )<br>- Llama 3 8B Instruct (GGUF Q4 <sub>0</sub> ) |
| <b>LLM Configuration</b> | Context Window: 1000 tokens, Temperature: 0.3                                                                                                                                                                         |
| <b>Vector Database</b>   | ChromaDB was used for storing and retrieving API fragments and contextual information.                                                                                                                                |
| <b>Observability</b>     | Langfuse was integrated for comprehensive monitoring and logging of the test generation and execution process.                                                                                                        |

easily adopted by other researchers in the field of API security testing.

### 1) HARDWARE AND SOFTWARE ENVIRONMENT

Our evaluation of Karate-BOLA-Guard was conducted using a typical commercial hardware setup, complemented by a carefully selected suite of open-source and popular software tools. This approach reflects real-world constraints while leveraging suitable technologies for our experiments. The following table outlines our experimental environment, detailing the hardware specifications, software stack, LLM configurations, and supporting technologies used:

### 2) TARGET API SELECTION

For our experiments, we selected three well-known intentionally vulnerable applications: VAmPI,<sup>2</sup> CrAPI,<sup>3</sup> and OWASP Juice Shop.<sup>4</sup> Each application presents distinct architectural patterns that enrich our testing scenarios. VAmPI offers a monolithic Flask API with straightforward vulnerability implementations, making it ideal for baseline testing. CrAPI employs a microservice-based architecture that reflects modern enterprise applications, allowing us to test our framework against distributed systems. Juice Shop provides a modern Node.js application with REST API endpoints, representing contemporary web application architectures. From a vulnerability perspective, all three applications feature comprehensive BOLA scenarios, though each implements different variations of authorization flaws. This diversity in implementation allows us to validate our framework's ability to detect BOLA vulnerabilities across various contexts and complexity levels. The applications maintain exemplary documentation of their API structures and implemented

<sup>2</sup>VAmPI: <https://github.com/erev0s/VAmPI>. - A deliberately vulnerable API implemented in Flask, chosen for its focused coverage of OWASP API Security Top 10 vulnerabilities

<sup>3</sup>CrAPI: <https://github.com/OWASP/crAPI> - A complex, microservice-based vulnerable application maintained by OWASP, selected for its enterprise-grade architecture and comprehensive vulnerability coverage.

<sup>4</sup>Juice Shop: <https://github.com/juice-shop/juice-shop> - One of the most complete deliberately vulnerable web applications, chosen for its modern tech stack and real-world scenarios.

security flaws, facilitating accurate validation of our test results. The following characteristics make these applications particularly suitable for our study:

- Comprehensive vulnerability coverage with varied BOLA implementations;
- Well-documented API structures and known vulnerabilities;
- Active maintenance with regular security patches and updates;
- Strong community support and involvement;
- Open-source nature enabling full reproducibility of experiments.

This diverse selection of target applications allows us to evaluate our framework's effectiveness across different technological stacks, architectural patterns, and implementation complexities. By testing against these three distinct applications, we can better assess the robustness and adaptability of our security testing approach in various real-world scenarios.

### 3) EXTENDED API TARGET SELECTION

To expand the scope of our evaluation and validate the framework's generalizability, we conducted additional experiments using a diverse set of APIs beyond our controlled test environment. In addition to our in-depth analysis of intentionally vulnerable applications, we extended our evaluation to include ten production APIs selected from APIRank [25], a comprehensive repository of OpenAPI Specification (OAS) files.

The selected APIs represent various domains and complexity levels, including:

- Social media platforms (Instagram);
- Version control systems (GitHub, GitLab);
- Transportation services (London Transport, Uber);
- Financial services (Yahoo Finance, Stripe);
- Content platforms (Medium);
- E-commerce solutions (Salesforce Partners and Customers, BigCommerce);
- Communication services (Telegram);
- Information retrieval systems (News Search Client).

This diverse selection enabled us to evaluate our framework's performance across different API architectures, authentication mechanisms, and business domains and with different configurations in terms of Model and configuration (GPU/CPU).<sup>5</sup>

This expansion of our experimental scope provides valuable insights into the framework's adaptability and performance across diverse API landscapes, though with the aforementioned constraints regarding production deployment testing.

<sup>5</sup>Due to ethical considerations and potential security implications, we limited our evaluation to test case generation and structural analysis without executing the tests against production endpoints. Additionally, it should be noted that the OAS specifications obtained from APIRank may not represent the complete API surface area and could potentially be outdated. The complete dataset of generated tests and analysis results is available in our project repository for transparency and reproducibility.

### C. IMPLEMENTATION OF KARATE-BOLA-GUARD

The implementation of Karate-BOLA-Guard involved integrating several key components to create a cohesive and efficient system for API security testing. Each component was carefully selected and integrated to address specific aspects of the test generation and execution process. The following details our integration process and the rationale behind each component choice:

- **LLM Service:** We employed GPT4ALL as our LLM wrapper, enabling local execution of the models. This decision was driven by several factors:
  - Local execution ensures data privacy and reduces latency in test generation;
  - GPT4ALL's compatibility with multiple models provided flexibility in our experimental setup;
  - The wrapper's straightforward API facilitated smooth integration into our existing pipeline.
- **Vector Database:** Chroma DB was selected for storing and retrieving API fragments and historical test data. Key considerations included:
  - Chroma DB's scalability in handling large volumes of embeddings efficiently;
  - Its optimized search algorithms, providing fast retrieval times crucial for real-time test generation;
  - Seamless integration with Python ecosystems, particularly LangChain.
- **RAG Implementation:** We utilized LangChain to implement the RAG-enhanced test generation process. This choice was based on:
  - LangChain's modular and extensible architecture, allowing easy integration of various LLMs and tools;
  - Built-in support for RAG techniques, streamlining our implementation process;
  - Robust community support and regular updates, ensuring ongoing improvements and bug fixes.
- **Testing Framework:** Karate was chosen as our testing framework due to:
  - Its intuitive syntax, which closely resembles natural language, making it ideal for LLM-generated test cases;
  - Built-in support for API testing and ability to handle complex scenarios;
  - Extensive documentation and active community support.
- **Observability:** Langfuse was integrated for comprehensive monitoring and analysis, offering:
  - Detailed tracing of LLM calls, including prompts and completions;
  - Performance metrics tracking, such as token usage and generation time;
  - Customizable logging and visualization capabilities.

### 1) INTEGRATION PROCESS

The integration of these components involved several steps and presented unique challenges:

- 1) **Prompt Engineering:** The process involved multiple, carefully designed calls to the LLM Service. Each prompt was meticulously crafted to include relevant context and precise instructions. Our aim was to elicit outputs that accurately incorporated Karate DSL Scenario elements, such as the "Given", "When", and "Then" keywords. Additionally, we focused on ensuring the inclusion of API-specific elements, including the correct URL, appropriate request method, and expected response. This detailed approach to prompt engineering was crucial in generating test scenarios that were both syntactically correct and contextually relevant to the API under test and relevant towards BOLA-oriented vulnerability discovery;
- 2) **Data Flow Optimization:** Efficient data transfer between the vector database, LLM service, and testing framework was crucial;
- 3) **Error Handling and Resilience:** Robust error handling mechanisms were implemented to manage potential failures in any component, ensuring the stability of the overall system;
- 4) **Scalability Considerations:** We designed the framework with scalability in mind, allowing for easy integration of additional LLMs or alternative vector storage solutions in future iterations.

### 2) LLM SELECTION AND CONFIGURATION

In alignment with our commitment to open-source contribution and experimental replicability, we selected four popular Open-Source LLMs for our study:

- Mistral Instruct 7B (GGUF Q4\_0 Quantization);
- Wizard v1.2 13B (GGUF Q4\_0 Quantization);
- Llama 3 8B Instruct (GGUF Q4\_0 Quantization);
- Phi-3 Mini Instruct (GGUF Q4\_0 Quantization).

These models were chosen based on their open-source nature, accessibility, and compatibility with various LLM wrappers. We employed the GGUF Q4\_0 quantization for all models to ensure consistency and broad compatibility. While this 4-bit quantization may impact model performance, a detailed analysis of the trade-offs between model size and output quality is beyond the scope of this study and presents an opportunity for future research.

For our experiments, we standardized the configuration across all models with a context window of 1000 tokens and a temperature setting of 0.3. The choice of a 1000-token context window was made to accommodate the limitations of most contemporary LLMs while still providing sufficient context for our test generation tasks. The temperature setting of 0.3 was selected to strike a balance between output creativity and deterministic behavior, favoring more predictable and focused test case generation.

These parameter choices were made based on our current understanding of LLM behavior in similar tasks. However, we acknowledge that optimal settings may vary depending on the specific requirements of API security testing. As such, future research could explore the impact of different context window sizes and temperature settings on the quality and effectiveness of generated test cases.

It is worth noting that as LLM technology rapidly evolves, future studies may benefit from larger context windows and more nuanced temperature control. Our configuration choices provide a baseline for comparison and a starting point for further optimization in the field of LLM-driven API security testing.

#### D. DATA COLLECTION AND PREPARATION

The foundation of our knowledge base rests on a carefully curated Karate-BOLA-DSL oriented dataset, specifically designed for BOLA-test-case context [26]. To enhance our framework's capability in syntax compatibility and verification steps, we incorporated the Karate Labs GitHub repository<sup>6</sup> into our knowledge base. This integration provides a comprehensive reference for Karate DSL syntax and best practices. These datasets are uploaded to the Vector Data Storage, and used later as context, for generating and validating karate tests cases.

#### E. API SPECIFICATION PARSING

Recognizing the limitations imposed by constrained context windows and the potential for extensive API specifications, particularly in larger applications beyond microservices, we developed a strategic approach to API specification parsing. This step is crucial for effective data management and processing within our framework.

Our solution involves a custom-built OAS Parser component, implemented as a Python script. This parser efficiently breaks down large API specifications into manageable chunks, a common practice in data management for handling substantial datasets. These parsed chunks are then systematically sent to our Vector Data Storage, implemented using ChromaDB.

We've designed our storage system with scalability and organization in mind. Each API's parsed data is stored in a separate collection within ChromaDB, with the collection name corresponding to the specific API. This approach allows our framework to handle multiple APIs concurrently, storing their respective data in distinct collections within the same ChromaDB instance. Such a design not only facilitates efficient data retrieval but also provides a clear structure for managing and accessing API-specific information throughout the test generation process.

This modular approach to data collection and preparation ensures that our framework can adapt to various API sizes and complexities, from small microservices to large-scale applications. It also sets the stage for future expansions,

allowing easy integration of additional APIs into our testing ecosystem without necessitating significant changes to the underlying data management structure.

#### F. TEST CASE GENERATION PROCESS

With the knowledge base data in place and configuration details set in the config file for the RAG System, our test case generation process can be initiated. The process follows these steps for each API fragment:

- 1) **Initial Karate Test Generation:** The system uses a predefined prompt to query the Vector Database for BOLA-related API fragments. Combining these with BOLA context examples and key elements from the API Fragment (such as URI or other relevant data), the LLM generates an initial test case. This test case is then stored as a feature file in the designated output folder.
- 2) **Karate Compatibility Verification:** The system takes the generated feature file as context, along with additional context from the Vector Database. It then queries the Karate DSL collection for relevant examples to check for appropriate use cases. A specialized prompt targets compatibility checking and test enhancements.
- 3) **Karate Syntax Verification:** The output from the previous step serves as context, along with similar or identical data retrieved from the Vector Database. This time, the system uses a prompt specifically designed for syntax validation.
- 4) **Runner Step:** The verified feature files are moved to a separate directory, which is mounted as a volume in a Docker container. This container includes the necessary data to run the tests in an isolated environment against the target service. The system generates both a raw text report for processing and uploading to the Vector Database, and an HTML report that can be automatically opened in a browser for user inspection.
- 5) **Error Check Loop:** To address interoperability issues, the system performs a predefined number of error check iterations (set to three in our experiments). When syntax or other errors occur, the system makes a new call to the LLM service. It retrieves error information for the specific generated test case from the Vector Database and uses this in a crafted prompt aimed at addressing syntax errors and other relevant issues. The previous step runs again, and if the error persists it runs throw this error check loop.
- 6) **Observability:** Throughout this process, the system makes calls to the Langfuse service, transmitting various data such as token count, LLM name, input, output, and any exceptions that occur. Users can access this service via a web browser to observe the behavior of the current run.

This systematic approach ensures a thorough and iterative test case generation process, with built-in verification steps

<sup>6</sup><https://github.com/karatelabs/karate>



and error handling to maximize the quality and relevance of the generated tests.

### G. EXPERIMENTAL PROCEDURE

To rigorously evaluate the efficacy of our Karate-BOLA-Guard framework, we devised a comprehensive experimental procedure that leverages the power of LLMs, in a structured, iterative manner. This methodology not only allows for the generation of sophisticated API security tests but also incorporates multiple verification and refinement stages, ensuring the quality and relevance of the produced test cases. The following outlines our systematic approach, which combines initial test generation with rigorous verification and error-checking processes::

- 1) **Initial Karate Test Generation (1 call):** Using BOLA context and API Fragment;
- 2) **Karate Compatibility Verification (1 call):** Using Karate DSL context and Karate context examples;
- 3) **Karate Syntax Verification (1 call):** Using Karate DSL context and syntax fixing instructions;
- 4) **Error Check Loop(Optional, 3 calls):** Three iterations of error checks, using corresponding test errors from the test report is run, if the generated test fails.

This procedure was repeated for each of the LLM (Mistral 7B, Wizard 13B, Phi-3 Mini Instruct, and Llama 3 8B) to ensure a comprehensive evaluation of our framework's performance across different model architectures.

## IV. DISCUSSION

The results of our experiments with Karate-BOLA-Guard provide valuable insights into the potential of RAG-enhanced, LLM-driven approaches for API security testing, particularly in the context of BOLA vulnerabilities. This section interprets our findings, discusses their implications, addresses limitations, and proposes future research directions.

### A. ANALYSIS OF SECURITY TEST CASE QUALITY IN API TESTING

This analysis examines a collection of security test cases focusing on authentication and authorization testing, identifying common patterns, issues, and areas for improvement. The test cases were written in Karate DSL and demonstrated both strengths and weaknesses in their implementation.

#### 1) SECURITY TEST SCENARIOS AND EVALUATION

Our initial evaluation was done on the tree experimental use cases VAmPI, Crapi, and OWASP Juice Shop, however, we extended our research to evaluate more production APIs, and based on model performance findings as shown in Section IV-C, we determined that GPU execution would be more appropriate for our evaluation due to:

- Significantly reduced execution times enabling more efficient testing cycles;

- Consistent performance characteristics across test iterations;
- Better representation of real-world deployment scenarios;
- More efficient resource utilization for time-constrained testing.

Furthermore, considering the balance between performance and output quality, we selected Llama 3 8B as our primary model for the extended API evaluation, despite its longer processing times, due to its more comprehensive output generation and consistently higher accuracy scores in our previous evaluations.

#### 2) STRUCTURAL ISSUES

The most prevalent issues were related to syntax and structure:

- **Incorrect Format Mixing:** Tests frequently mixed JSON/YAML syntax with Karate DSL. An example:

```
‘‘When method post with body {‘username
  ‘: ‘admin’, ‘password’: ‘’}’’
```

Should be written as:

```
And request {
  username: ‘admin’,
  password: ‘’
}
When method post
```

- **Missing Essential Elements:** Many tests lacked proper Feature declarations or had incorrect Background sections:

```
feature: Broken Authentication /
  Authorization # Incorrect
Feature: Broken Authentication /
  Authorization # Correct
```

#### 3) POSITIVE ASPECTS

Several significant strengths were identified in the test cases:

- 1) **Security Focus:** Tests consistently demonstrated robust verification of critical security aspects:
  - Authentication validation with comprehensive token handling;
  - Multi-level authorization checks;
  - Error message information disclosure analysis;
  - Advanced injection attack detection targeting auth endpoints;
  - BOLA-specific exploit pattern recognition.
- 2) **Comprehensive Scenarios:** The tests exhibited sophisticated coverage through multiple approaches:
  - Multiple authentication states testing:
 

```
Scenario: Verify login endpoint {–}
  empty password
Scenario: Verify login endpoint {–}
  wrong password
```
  - Advanced fuzzing implementations with parameterized inputs;

**TABLE 2.** Evaluation results by factor of generated test cases. Component descriptions and scoring calculations detailed in section IV-B4.

| TraceID          | API               | Accuracy Components |       |       |       | A   | Complexity Components |       |       |       | C   | Interop. Components |       |       |       | I   |
|------------------|-------------------|---------------------|-------|-------|-------|-----|-----------------------|-------|-------|-------|-----|---------------------|-------|-------|-------|-----|
|                  |                   | $a_1$               | $a_2$ | $a_3$ | $a_4$ |     | $c_1$                 | $c_2$ | $c_3$ | $c_4$ |     | $i_1$               | $i_2$ | $i_3$ | $i_4$ |     |
| L3I-tel-241c832d | Telegram          | 0.7                 | 0.7   | 0.3   | 0.7   | 3.4 | 0.7                   | 0.7   | 0.7   | 0.7   | 3.8 | 0.7                 | 0.7   | 0.7   | 0.7   | 3.8 |
| L3I-ube-86b368f6 | uber              | 0.7                 | 0.7   | 0.3   | 0.7   | 3.4 | 0.7                   | 0.7   | 0.7   | 0.7   | 3.8 | 0.7                 | 0.7   | 0.7   | 0.7   | 3.8 |
| L3I-tra-0178f6dc | tfl               | 0.8                 | 0.7   | 0.5   | 0.6   | 3.6 | 0.8                   | 0.7   | 0.6   | 0.5   | 3.6 | 0.9                 | 0.7   | 0.8   | 0.7   | 4.1 |
| L3I-sla-df02876a | slack             | 0.9                 | 0.8   | 0.7   | 0.8   | 4.2 | 0.9                   | 0.8   | 0.7   | 0.7   | 4.1 | 0.9                 | 0.8   | 0.8   | 0.8   | 4.3 |
| L3I-sal-95443bb1 | sf cus-<br>tomer  | 0.7                 | 0.6   | 0.5   | 0.5   | 3.3 | 0.7                   | 0.6   | 0.5   | 0.4   | 3.2 | 0.6                 | 0.7   | 0.6   | 0.6   | 3.5 |
| L3I-sal-55f44e62 | sf<br>partner     | 0.8                 | 0.8   | 0.7   | 0.6   | 3.9 | 0.8                   | 0.7   | 0.6   | 0.7   | 3.8 | 0.9                 | 0.8   | 0.8   | 0.7   | 4.2 |
| L3I-new-9845415b | news              | 0.8                 | 0.7   | 0.7   | 0.6   | 3.8 | 0.7                   | 0.8   | 0.6   | 0.6   | 3.7 | 0.8                 | 0.8   | 0.7   | 0.7   | 4   |
| L3I-med-3755e4f7 | medium            | 0.7                 | 0.6   | 0.6   | 0.6   | 3.5 | 0.8                   | 0.7   | 0.6   | 0.5   | 3.6 | 0.7                 | 0.7   | 0.8   | 0.6   | 3.8 |
| L3I-jui-5f2a1c4c | juiceshop         | 0.9                 | 0.8   | 0.8   | 0.7   | 4.2 | 0.9                   | 0.8   | 0.8   | 0.7   | 4.2 | 0.8                 | 0.8   | 0.9   | 0.8   | 4.3 |
| L3I-ins-bb06bdb7 | instagram         | 0.9                 | 0.8   | 0.8   | 0.8   | 4.3 | 0.9                   | 0.9   | 0.8   | 0.8   | 4.4 | 0.9                 | 0.8   | 0.9   | 0.8   | 4.4 |
| L3I-git-de39e840 | gitlab            | 0.9                 | 0.8   | 0.8   | 0.7   | 4.2 | 0.9                   | 0.8   | 0.8   | 0.7   | 4.2 | 0.8                 | 0.7   | 0.9   | 0.8   | 4.2 |
| L3I-git-f97a3bbc | github            | 0.7                 | 0.7   | 0.6   | 0.7   | 3.7 | 0.7                   | 0.7   | 0.7   | 0.7   | 3.8 | 0.5                 | 0.6   | 0.7   | 0.7   | 3.5 |
| L3I-big-1fcae30c | big-<br>ecommerce | 0.7                 | 1     | 0.7   | 0.7   | 4.1 | 1                     | 0.7   | 1     | 0.7   | 4.4 | 0.7                 | 0.7   | 0.8   | 1     | 4.2 |
| L3I-yah-7676d8f3 | yahoo             | 0.3                 | 0.7   | 0.7   | 0.3   | 3   | 0.3                   | 0.7   | 0.7   | 0.3   | 3   | 0.3                 | 0.7   | 0.3   | 0.3   | 2.6 |
| L3I-vam-38f41c4b | vampi             | 0.7                 | 0.8   | 0.5   | 0.6   | 3.6 | 0.9                   | 0.8   | 0.4   | 0.6   | 3.7 | 0.9                 | 0.8   | 0.8   | 0.8   | 4.3 |
| L3I-str-24998edb | stripe            | 0.9                 | 0.8   | 0.8   | 0.6   | 4.1 | 0.8                   | 0.9   | 0.6   | 0.5   | 3.8 | 0.9                 | 0.8   | 0.7   | 0.8   | 4.2 |
| L3I-spo-b735d478 | spotify           | 0.6                 | 0.7   | 0.5   | 0.4   | 3.2 | 0.6                   | 0.7   | 0.5   | 0.3   | 3.1 | 0.8                 | 0.6   | 0.5   | 0.4   | 3.3 |
| L3I-cra-1ce589cd | craip             | 0.5                 | 0.4   | 0.3   | 0.4   | 2.6 | 0.5                   | 0.4   | 0.3   | 0.2   | 2.4 | 0.5                 | 0.4   | 0.3   | 0.3   | 2.5 |

- Custom assertion patterns for detailed response validation;
- Sophisticated data manipulation for edge case testing;
- Combined attack vectors including SQL injection and XSS targeting auth flows.

#### 4) AREAS FOR IMPROVEMENT

Our analysis identified several recurring patterns where test implementation could be enhanced for greater effectiveness and reliability. These opportunities for improvement span multiple aspects of test design, including code structure, authentication handling, test logic, and assertion implementation. The following sections detail key areas where systematic improvements would strengthen the overall quality of generated test cases.

- **Authentication Handling:** Enhance token and session management:
  - Standardize token management across test suites;
  - Implement proper session cleanup;
  - Add validation for various authentication states.
- **Consistent Format:** Maintain proper Karate DSL syntax throughout:

```
Feature: Authentication Security
  Tests
  Background:
    * url baseUrl
  Scenario: Authentication Validation
    Given request payload
    When method post
    Then status 401
    And match response.message == '#
    regex ^Authentication.*'
```

- **Clear Assertions:** Use explicit assertions with proper matching syntax:

```
# Incorrect
Then status code is 401

# Correct
Then status 401
And match response.message == '#regex
^Authentication.*'
```

- **Test Data Management:** Improve isolation and cleanup:
  - Implement proper data separation between test scenarios;
  - Add cleanup steps after test execution;
  - Use dynamic test data generation where appropriate.
- **Error Scenario Coverage:** Expand error condition handling:
  - Add comprehensive validation for authorization failures;
  - Include edge cases in error scenarios;
  - Implement consistent error response validation.

These areas for improvement highlight the need for more robust test generation patterns in our framework, particularly focusing on data management, authentication flows, and error handling. While our current implementation demonstrates strong security coverage, enhancing these aspects would lead to more maintainable and reliable test suites. Addressing these improvements could significantly enhance the framework's effectiveness in detecting BOLA vulnerabilities while reducing false positives.

**TABLE 3.** Evaluation results for Llama 3 8B model across different APIs.

| TraceID                                      | API                 | A   | C   | I   | Notes                                                                                                                                                                                                                                                                                   |
|----------------------------------------------|---------------------|-----|-----|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| L3I-tel-241c832d-8d41-44c3-95d1-94dfb1b38711 | telegram            | 3.4 | 3.8 | 3.8 | Includes both OAuth exploit attempt and role-based access verification. Good validation structure but could benefit from helper functions for generating payloads instead of generic token placeholders.                                                                                |
| L3I-ube-86b368f6-280f-47cb-a208-10f9475cb181 | uber                | 3.4 | 3.8 | 3.8 | Basic authentication testing for time estimates endpoint. Includes external feature integration for unauthenticated access verification. Simple test structure with limited error scenarios and basic response validation.                                                              |
| L3I-tra-0178f6dc-2107-4460-bc6f-ed818a9184d9 | london transport    | 3.6 | 3.6 | 4.1 | Advanced fuzzing implementation with nested loops and security-oriented test data. Combines authorization testing with injection attempts. Well-structured test vectors for multiple attack patterns.                                                                                   |
| L3I-sla-df02876a-bc29-41e0-96b2-d01f77e99f09 | slack               | 4.1 | 4.1 | 4.3 | Comprehensive auth testing with multiple scenarios covering unauthorized, invalid credentials, role verification, and success paths. Well-structured response validation..                                                                                                              |
| L3I-sal-95443bb1-852e-4f49-ab4f-f94ec1121c63 | salesforce-customer | 3.3 | 3.2 | 3.5 | Basic authentication testing with limited scenarios covering only token validation. Simple response validation with minimal assertion complexity. Could benefit from additional error cases and more detailed response verification.                                                    |
| L3I-sal-55f44e62-c5b0-4f51-b740-eb5a92aef66c | salesforce-partner  | 3.9 | 3.8 | 4.2 | Basic API key authorization testing with three negative test cases. Simple endpoint verification with minimal validation complexity. Focuses on unauthorized access states but lacks positive test cases.                                                                               |
| L3I-new-9845415b-d659-4347-a21e-da5b86eb67a9 | news-search-client  | 3.8 | 3.7 | 4   | Comprehensive authentication testing covering multiple auth mechanisms (refresh tokens, API keys, OAuth). Well-structured scenarios with proper error validation and status code checks. Strong coverage of different authorization failure modes with appropriate response validation. |
| L3I-med-3755e4f7-d851-46c7-8868-d4e46cf4536f | medium              | 3.5 | 3.6 | 3.8 | Role-based access control testing using scenario outline for different user roles. Includes external feature file calls and proper assertion checks. Good structured testing approach with reusable components, though could benefit from more detailed response validation.            |
| L3I-jui-5f2a1c4c-2b4e-4543-ab5b-78184aa33d66 | juiceshop           | 4.2 | 4.2 | 4.3 | Advanced security testing combining OAuth token validation, fuzzing, and refresh token abuse. Implements comprehensive attack simulation with SQL injection, XSS vectors, and malicious redirects. Strong security patterns with nested iteration for thorough vulnerability testing.   |
| L3I-ins-bb06bdb7-d8d1-406e-93c9-c2f9f0e2d0b2 | instagram           | 4.3 | 4.4 | 4.4 | Combined testing of user enumeration and OAuth token security. Good coverage of unauthorized access patterns and OAuth exploitation attempts. Includes both positive and negative test cases with appropriate response validation.                                                      |
| L3I-git-de39e840-ca55-4183-b8eb-1551a3e37b0c | gitlab              | 4.2 | 4.2 | 4.2 | Advanced OAuth security testing with comprehensive fuzzing strategy. Iterative testing of multiple attack vectors including SQL injection, XSS, and redirect manipulation. Strong validation logic with both structural and security-focused assertions.                                |
| L3I-git-f97a3bbc-14f5-45c5-80ad-1af29e2087b3 | github              | 3.7 | 3.8 | 3.5 | Covering multiple authentication mechanisms (OAuth, Bearer tokens, API keys). Well-structured scenarios testing different authorization levels and proper error handling. Strong validation of access control with response checks.                                                     |
| L3I-big-1fcae30c-aae9-4e46-bf54-b151ee5a9aee | big-e-commerce      | 4.1 | 4.4 | 4.2 | Advanced security testing with multiple vectors: OAuth exploitation, account enumeration, and fuzzing. Uses generic mock values instead of realistic data patterns.                                                                                                                     |
| L3I-yah-7676d8f3-b015-4b66-9826-949adff48a50 | yahoo               | 3   | 3   | 2.6 | Security testing combining OAuth flows and fuzzing but with some syntax issues. Incorrect method usage (GET for token operations) and redundant URL/path definitions. Implementation needs improvement in request structure and HTTP method selection.                                  |
| L3I-vam-38f41c4b-ea54-476b-91a6-6550982dec2a | vampi               | 3.6 | 3.7 | 4.3 | Well-structured login flow testing with session validation. Good sequential testing of authentication states and token handling. Includes proper response validation and error checking for both positive and negative scenarios.                                                       |
| L3I-str-24998edb-33a0-431a-ab09-379906511490 | stripe              | 4.1 | 3.8 | 4.2 | Comprehensive token validation testing with extensive coverage of different token states (expired, revoked, tampered, stolen). Consistent validation patterns and proper response checking. Strong focus on security edge cases including token reissuance scenarios.                   |
| L3I-spo-b735d478-03c4-4841-bdbc-583caa5ac679 | spotify             | 3.2 | 3.1 | 3.3 | OAuth security testing covering token generation, access control, and refresh token abuse. Includes external feature file integration and conditional logic, but uses generic token placeholder.                                                                                        |
| L3I-cra-1ce589cd-6a5f-498e-915d-81aec581be97 | crapi               | 2.6 | 2.4 | 2.5 | Basic authentication testing with inconsistent validation patterns. Missing proper response validation and error handling.                                                                                                                                                              |

**Note:** This table presents evaluation metrics for the Llama 3 8B Model only. All metrics are scored using the evaluation metrics as defined in Section II-F on a scale of 1-5, where 5 represents the highest performance. TraceIDs are preserved for cross-reference with the complete dataset in the project repository (See Section VI).

## B. INTERPRETATION OF RESULTS

### 1) COMPARISON WITH PREVIOUS APPROACHES

Our research has explored various methodologies for generating Karate DSL security-oriented tests using LLMs. This section provides a comparative analysis of our previous approaches and highlights the advancements achieved in our

current methodology, with a focus on the performance of Mistral 7B, Wizard v1.2 13B, Phi-3 Mini, and Llama 3 8B Instruct models.

### 2) ONE-SHOT PROMPT APPROACH

Initial experiments utilized one-shot prompts with LLMs to generate test cases. While this approach demonstrated the

potential of LLM-assisted test generation, it exhibited several limitations in efficiency, accuracy, and consistency [24].

### 3) BOLA-TRAINED MODEL WITH ONE-SHOT PROMPTS

Building upon our initial findings [24], we developed a model specifically trained on BOLA vulnerabilities. This approach yielded notable improvements in relevance, efficiency, and specialization for BOLA-specific security concerns. However, it still failed to achieve fully autonomous test generation [27].

### 4) PERFORMANCE ACROSS LLM MODELS

The radar charts in Figure 3 demonstrate distinct performance patterns across our primary case study APIs and models. The following evaluation metrics and scoring calculations are used consistently throughout our analysis, including in Tables 2 and 4<sup>7</sup>:

- **Accuracy** ( $a_1 - a_4$ ): Measuring BOLA Pattern Coverage, Authorization Boundary Testing, Edge Case Handling, and Validation Completeness;
- **Complexity** ( $c_1 - c_4$ ): Evaluating Scenario Structure, Authorization Logic, Data Manipulation, and Assertion Coverage;
- **Interoperability** ( $i_1 - i_4$ ): Assessing Syntax Correctness, Authentication Integration, Framework Compatibility, and Data Handling.

Final scores for each category are calculated using:

$$A = 1 + \sum_{i=1}^4 a_i \quad C = 1 + \sum_{i=1}^4 c_i \quad I = 1 + \sum_{i=1}^4 i_i$$

where each component is scored from 0 to 1, resulting in final scores ranging from 1 to 5.

A detailed analysis can be seen in Tables 4. These patterns are further validated in Table 2 and 3 against Llama 3 8B model, and can be visualised in Figure 4, where it shows consistent performance metrics across an extended collection of test APIs.<sup>8</sup>

Our detailed analysis regarding Model-Specific, API-Specific Characteristics and Metric Relationships can be observe in the below:

#### 1) Model-Specific Performance:

- *Llama 3 8B* demonstrates the most consistent performance across all APIs (Accuracy: 3.1-3.4, Interoperability: 3.7-4.3), with particularly strong interoperability metrics in JuiceShop;
- *Wizard v1.2* shows moderate performance (Accuracy: 2.7-3.2, Interoperability: 2.7-3.7), with notable variation across different APIs;

- *Mistral* exhibits API-dependent performance, achieving strong interoperability in Crapi (3.9) but lower metrics in JuiceShop;
- *Phi-3 Mini* maintains relatively consistent performance (Accuracy: 3.0-3.3, Interoperability: 3.2-3.8), showing particular stability in accuracy metrics.

#### 2) API-Specific Characteristics:

- *VAmPI* demonstrates balanced performance across metrics, with Llama 3 8B achieving highest scores (A: 3.3, C: 3.7, I: 4.1);
- *JuiceShop* shows the highest interoperability scores with Llama 3 8B (4.3), but lower complexity scores across most models;
- *Crapi* exhibits strong complexity scores with Llama 3 8B (3.8) and consistent interoperability across multiple models.

#### 3) Metric Relationships:

- *Interoperability* shows the widest range (2.7-4.3), with highest values achieved in JuiceShop testing;
- *Complexity* demonstrates moderate variation (2.2-3.8), with Llama 3 8B consistently achieving higher scores;
- *Accuracy* maintains the most stable range (2.7-3.4) across all models and APIs, suggesting a baseline level of test quality.

These observations indicate that while each model demonstrates specific strengths and capabilities in different testing contexts, Llama 3 8B Instruct consistently emerges as the most reliable and balanced performer across all three APIs and evaluation metrics. The optimal choice of model should be carefully influenced by the target API's specific characteristics, as can be seen in Table 5 summarizes these key performance patterns and relationships, providing a comprehensive overview of model behavior and effectiveness across all three APIs, which can guide practical implementation decisions in various testing contexts.

Building upon our comparative model analysis through radar charts (Figure 3) and aggregate metrics (Table 5), which demonstrated Llama 3 8B's consistent performance in accuracy (3.1-3.4) and strong interoperability metrics (3.7-4.3) across APIs and strong interoperability metrics, particularly in comparison to other models, Table 5 presents a granular analysis of test cases generated by Llama 3 8B Instruct running on GPU. This model was selected for detailed analysis due to its superior stability and balanced performance characteristics across all evaluation dimensions, as evidenced by our radar analysis.

Due to space constraints, we present the final evaluation scores accompanied by descriptive notes highlighting key characteristics that influenced the scoring. The complete test implementations, generation logs, token usage data, and related execution metrics are available in our public repository<sup>9</sup> for further examination and reproducibility purposes.

<sup>9</sup><https://github.com/EmilMarian/Karate-BOLA-Guard-Framework>

<sup>7</sup>In Tables 2 and 4 the TraceIDs have been truncated for readability.

<sup>8</sup>As detailed in Section IV-C, we focused our extended evaluation on the Llama 3 8B model with GPU acceleration, as it provided the optimal balance of performance (average Accuracy: 3.7, Interoperability: 3.9) and computational efficiency (average processing time: 143.76s) within our resource and time constraints.



TABLE 4. Detailed evaluation results by model and API. Uses same component descriptions and scoring calculations as detailed in Section IV-B4.

| TraceID          | API       | Model               | Accuracy Components |                |                |                | A   | Complexity Components |                |                |                | C   | Interop. Components |                |                |                | I   |
|------------------|-----------|---------------------|---------------------|----------------|----------------|----------------|-----|-----------------------|----------------|----------------|----------------|-----|---------------------|----------------|----------------|----------------|-----|
|                  |           |                     | a <sub>1</sub>      | a <sub>2</sub> | a <sub>3</sub> | a <sub>4</sub> |     | c <sub>1</sub>        | c <sub>2</sub> | c <sub>3</sub> | c <sub>4</sub> |     | i <sub>1</sub>      | i <sub>2</sub> | i <sub>3</sub> | i <sub>4</sub> |     |
| W12-vam-4a311daa | Vampi     | Wizard v1.2         | 0.7                 | 0.5            | 0.3            | 0.7            | 3.2 | 0.7                   | 0.7            | 0.5            | 0.7            | 3.6 | 0.4                 | 0.3            | 0.5            | 0.5            | 2.7 |
| MI-vam-b10391c7  | Vampi     | Mistral Instruct    | 0.5                 | 0.5            | 0.3            | 0.7            | 3   | 0.9                   | 0.5            | 0.5            | 0.6            | 3.5 | 0.7                 | 0.7            | 0.7            | 0.5            | 3.6 |
| L3I-vam-38f41c4b | Vampi     | Llama 3 8B Instruct | 0.7                 | 0.5            | 0.4            | 0.7            | 3.3 | 0.9                   | 0.7            | 0.5            | 0.6            | 3.7 | 0.9                 | 0.7            | 0.8            | 0.7            | 4.1 |
| P3M-vam-22f1fc7b | Vampi     | Phi-3 Mini Instruct | 0.7                 | 0.5            | 0.3            | 0.3            | 2.8 | 0.7                   | 0.7            | 0.5            | 0.5            | 3.4 | 0.9                 | 0.7            | 0.7            | 0.5            | 3.8 |
| W12-jui-clcc6761 | JuiceShop | Wizard v1.2         | 0.7                 | 0.5            | 0.5            | 0.3            | 3   | 0.7                   | 0.7            | 0.4            | 0.5            | 3.3 | 0.7                 | 0.7            | 0.5            | 0.7            | 3.6 |
| MI-jui-43c0b47a  | JuiceShop | Mistral Instruct    | 0.5                 | 0.5            | 0.1            | 0.5            | 2.6 | 0.5                   | 0.3            | 0.3            | 0.1            | 2.2 | 0.7                 | 0.7            | 0.5            | 0.3            | 3.2 |
| L3I-jui-5f2a1c4c | JuiceShop | Llama 3 8B Instruct | 0.7                 | 0.5            | 0.4            | 0.5            | 3.1 | 0.7                   | 0.7            | 0.4            | 0.5            | 3.3 | 0.7                 | 0.7            | 0.5            | 0.4            | 3.3 |
| P3M-jui-80e2aceb | JuiceShop | Phi-3 Mini Instruct | 0.7                 | 0.5            | 0.3            | 0.5            | 3   | 0.9                   | 0.7            | 0.3            | 0.3            | 3.2 | 0.7                 | 0.7            | 0.5            | 0.5            | 3.4 |
| W12-cra-eae55950 | Crapi     | Wizard v1.2         | 0.7                 | 0.4            | 0.3            | 0.3            | 2.7 | 0.7                   | 0.7            | 0.3            | 0.3            | 3   | 0.8                 | 0.7            | 0.7            | 0.5            | 3.7 |
| L3I-cra-1ce589cd | Crapi     | Llama 3 8B Instruct | 0.7                 | 0.7            | 0.3            | 0.7            | 3.4 | 0.9                   | 0.7            | 0.5            | 0.7            | 3.8 | 0.8                 | 0.7            | 0.7            | 0.5            | 3.7 |
| MI-cra-8c82a83e  | Crapi     | Mistral Instruct    | 0.7                 | 0.4            | 0.3            | 0.3            | 2.7 | 0.7                   | 0.7            | 0.3            | 0.3            | 3   | 0.9                 | 0.8            | 0.7            | 0.5            | 3.9 |
| P3M-cra-02fbdde3 | Crapi     | Phi-3 Mini Instruct | 0.7                 | 0.5            | 0.4            | 0.7            | 3.3 | 0.7                   | 0.5            | 0.4            | 0.3            | 2.9 | 0.7                 | 0.7            | 0.5            | 0.3            | 3.2 |

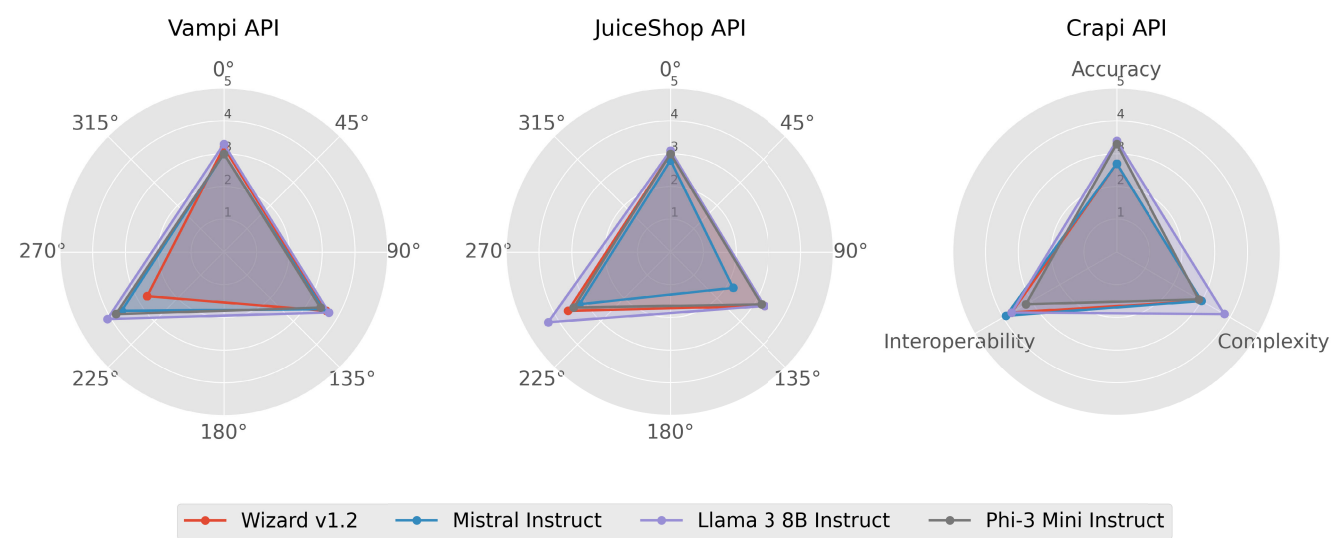


FIGURE 3. Performance metrics radar charts across different APIs and models.

*Duration Distribution Analysis:* The duration patterns illustrated in Figure 5 highlight the variability in test generation session durations across models and APIs. This analysis provides key insights into the temporal performance of our test generation framework, focusing on the time required to generate tests rather than their execution.

Our Llama 3 8B Instruct, exhibits the highest mean test generation duration (143.76s), while maintaining the best consistency across APIs (Coefficient of Variation (CoV): 13.20%). This stability suggests reliable performance across different API complexities, though at the cost of

longer generation times. The model’s narrow interquartile range (137s to 151s) further demonstrates its consistent performance characteristics.

Mistral Instruct achieves a more balanced performance profile, with a mean generation time of 72.14s and moderate variability (CoV: 18.05%). Its performance remains relatively stable across APIs, with durations typically ranging between 62.25s and 80.75s, making it a reliable choice for consistent test generation scenarios.

The smallest model, Phi-3 Mini Instruct, shows the fastest mean generation time (69.58s) but exhibits the highest

TABLE 5. Key performance patterns across different APIs and models.

| VAmPI                                                                                                                                                                                                                   | JuiceShop                                                                                                                                                                                       | Crapi                                                                                                                                                                                       | Overall Trends                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Llama 3 8B leads with balanced metrics (A: 3.3, C: 3.7, I: 4.1);</li><li>Good complexity scores across models;</li><li>Wizard v1.2 shows lowest interoperability (2.7).</li></ul> | <ul style="list-style-type: none"><li>Strongest interoperability with Llama 3 8B (4.3);</li><li>Lower complexity scores overall;</li><li>Consistent accuracy across models (2.8-3.1).</li></ul> | <ul style="list-style-type: none"><li>Highest complexity with Llama 3 8B (3.8);</li><li>Strong Mistral interoperability (3.9);</li><li>Most consistent accuracy scores (3.3-3.4).</li></ul> | <ul style="list-style-type: none"><li>Llama 3 8B shows best overall performance;</li><li>Interoperability varies significantly (2.7-4.3);</li><li>Accuracy remains most stable metric.</li></ul> |

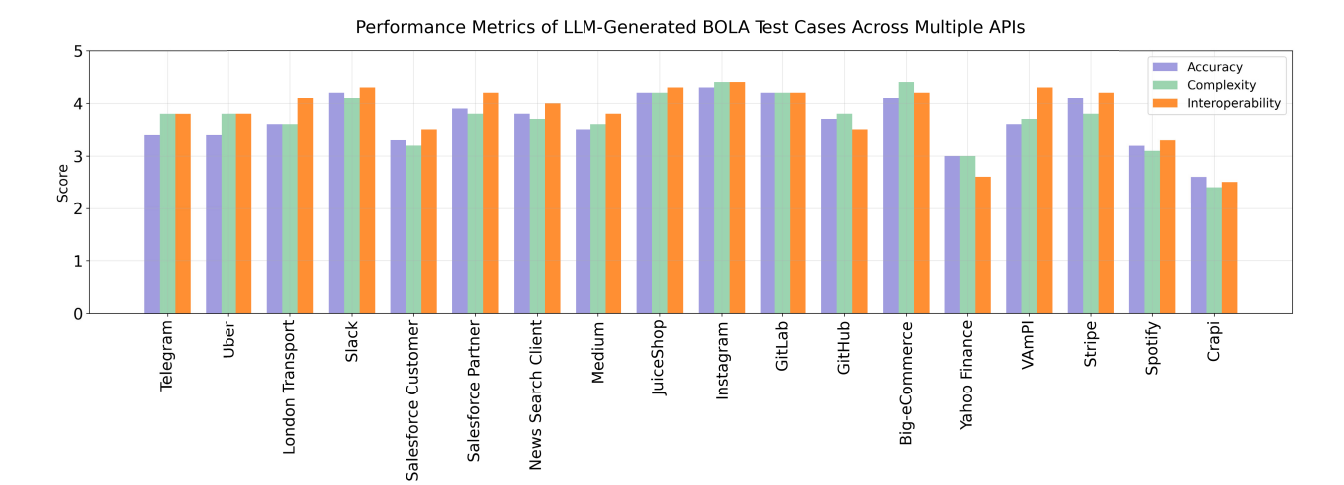


FIGURE 4. Evaluation metrics against extended research on multiple APIs on Llama 3 8B Model and GPU as compute device.

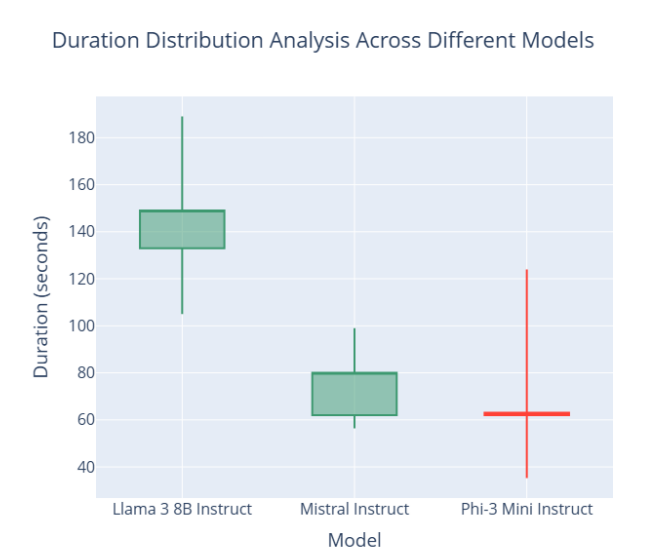


FIGURE 5. Test generation duration distribution analysis across different APIs and models (GPU).

variability (CoV: 42.45%). This significant variation in performance (interquartile range: 50.46s to 82.00s) suggests that while the model can achieve rapid test generation in optimal conditions, its performance is less predictable across different API contexts. This characteristic makes it

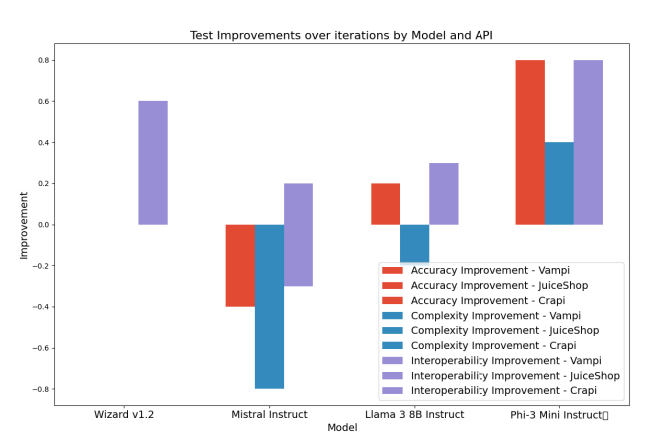


FIGURE 6. Metric Improvements across models and APIs.

suitable for rapid prototyping scenarios where execution time consistency is less critical.

The API-specific test generation durations reveal additional insights:

- VAmPI API:** Demonstrates consistent generation times with a mean around 145s (GPU), showing stable performance across test iterations.
- JuiceShop API:** Shows moderate variability in generation times with an average around 165s (GPU),

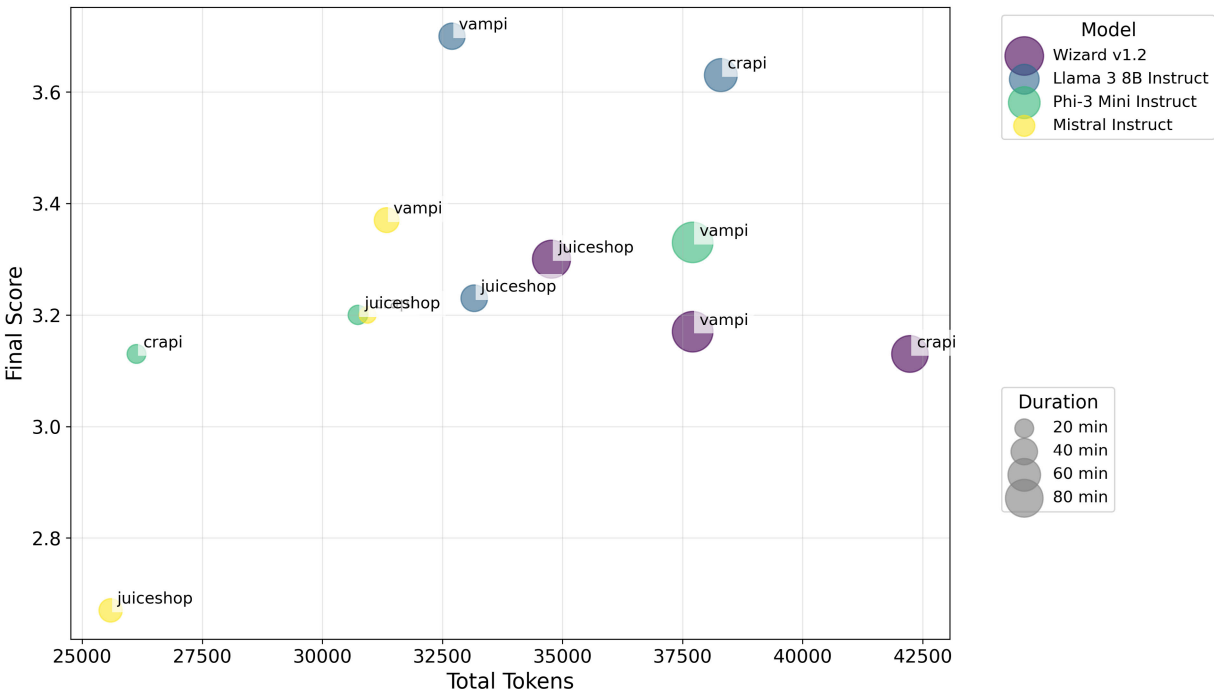


FIGURE 7. Resource Utilization analysis: Average duration vs token usage (CPU).

particularly evident when processing complex authentication scenarios.

- **Crapi API:** Exhibits the highest variability in generation times, ranging from 120s to 180s (GPU), suggesting that API complexity significantly influences generation duration.

The analysis underscores a correlation between model size and test generation characteristics:

- **Stability vs. Speed:** Larger models exhibit more consistent test durations but require substantially more time, while smaller models achieve rapid generation at the expense of increased variability.
- **Impact of API Complexity:** More complex APIs, such as JuiceShop, result in longer test generation durations, with the impact magnified in larger models.
- **Efficiency of Mid-Size Models:** Llama 3 8B demonstrates the optimal balance between test generation speed and stability across varying API complexities.

These findings suggest that selecting a model for test generation should depend on the specific requirements of the scenario, such as the complexity of the target API and the desired trade-off between generation speed and stability. Larger models are better suited for scenarios prioritizing consistency and predictability, whereas smaller models excel in contexts requiring rapid test generation iterations.

C. PERFORMANCE AND RESOURCE UTILIZATION ANALYSIS

Figures 6 and 7 present a comprehensive view of model performance improvements and resource utilization patterns across different APIs. The analysis reveals intricate relationships between performance metrics, computational resources, and efficiency characteristics of each model-API combination.<sup>10</sup>

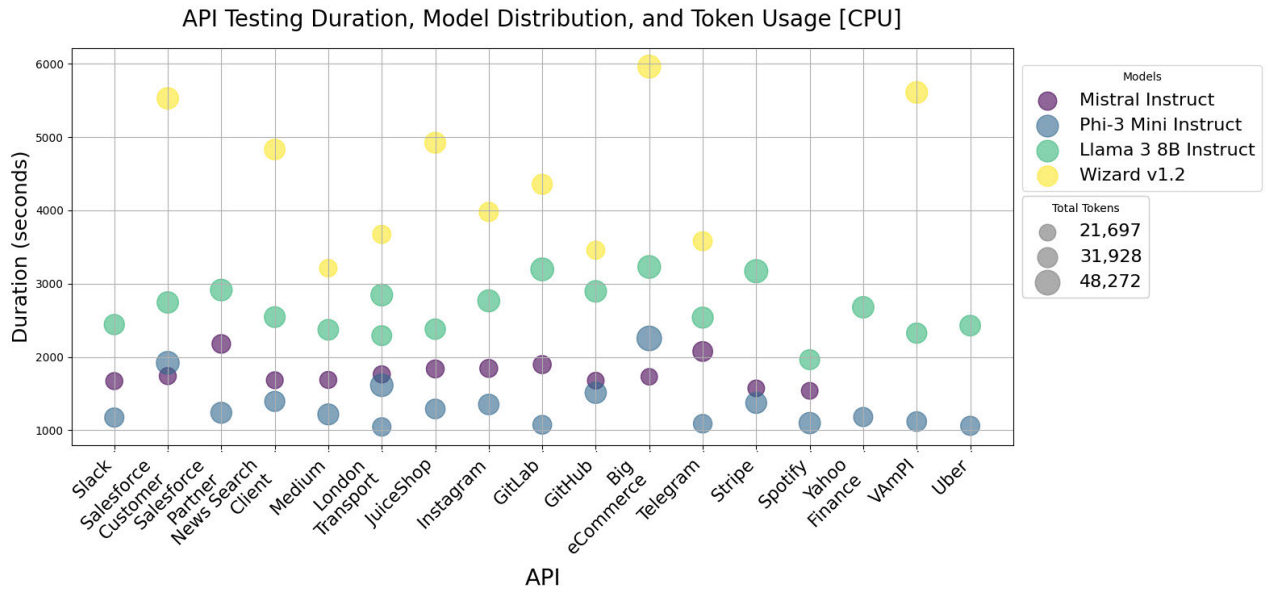
1) HARDWARE PERFORMANCE COMPARISON

Initial testing demonstrated a substantial performance differential between CPU and GPU execution:

- **GPU Execution:** Average processing times ranged from 69.58 seconds (Phi-3 Mini) to 143.76 seconds (Llama 3 8B);
- **CPU Execution:** Average processing times ranged from 1,333.78 seconds (Phi-3 Mini) to 4,463.45 seconds (Wizard v1.2).

This represents approximately a 20-25x performance improvement when utilizing GPU acceleration. Notably, the Wizard v1.2 model could only be executed on CPU due to VRAM constraints, highlighting the practical limitations of larger models in resource-constrained environments.

<sup>10</sup>The evaluation for the test generated using Wizard v1.2 Model, are based on the CPU configuration, as we weren't able to run it on GPU due to VRAM limitation.



**FIGURE 8.** A visualisation of the session duration, total token usage (input+output) while using CPU as compute device.

#### D. PERFORMANCE IMPROVEMENT PATTERNS

As can be observed in Figure 6, our analysis of metric improvements across different model-API combinations reveals distinct patterns when examining accuracy (A), interoperability (I), and complexity (C) metrics:

##### 1) WIZARD V1.2 PERFORMANCE

The model showed consistent performance across most scenarios, with notable improvements only in interoperability for the JuiceShop API (+0.60). Accuracy and complexity metrics remained unchanged ( $\Delta = 0$ ) across all APIs, suggesting a stable baseline performance for BOLA vulnerability detection.

##### 2) LLAMA 3 8B INSTRUCT PERFORMANCE

Llama demonstrated balanced improvements:

- JuiceShop API: Accuracy improved by +0.10;
- Crapi API: Interoperability increased by +0.20;
- VAmPI API: Interoperability showed +0.10 improvement.

These consistent, though modest, improvements suggest effective adaptation across different API contexts.

##### 3) MISTRAL INSTRUCT PERFORMANCE

Mistral showed the most varied results:

- JuiceShop API: Decreased performance across all metrics
  - Accuracy:  $-0.40$ ;
  - Complexity:  $-0.80$ ;
  - Interoperability:  $-0.30$ ;
- Crapi API: Improved accuracy by +0.20.

**TABLE 6.** Model Performance comparison on GPU.

| Model      | Avg. Duration (s) | Total Tokens | Input Tokens | Output Tokens |
|------------|-------------------|--------------|--------------|---------------|
| Llama 3 8B | 143.76            | 36,591       | 26,709       | 9,883         |
| Mistral    | 72.14             | 25,225       | 21,290       | 3,936         |
| Phi-3 Mini | 69.58             | 31,007       | 25,079       | 5,928         |

#### 4) KEY FINDINGS

- 1) The zero improvements ( $\Delta M = 0$ ) often is due to the fact that there is no notable differences between the initial test generated and the final one;
- 2) Interoperability improvements are most common, indicating that the iterative process primarily enhances syntax correctness ( $i_1$ ) and framework compatibility ( $i_3$ );
- 3) Negative changes in complexity metrics, particularly for Mistral, indicate that there are situation when the model generates a slightly degraded test and can be seen across test characteristics.

#### 5) TOKEN USAGE ANALYSIS

Figures 8 and 9 provide detailed visualizations of Duration and Token Usage across Models and APIs for CPU and GPU processing respectively. While token usage remains consistent within similar ranges (21k to 48k) for the same API-Model combinations, the primary difference lies in processing duration. Notably, despite variations in processing time, we observed no significant differences in the evaluation criteria of the generated tests.

*Resource Utilization Characteristics:* Figure 7 reveals distinct resource consumption patterns that correlate with



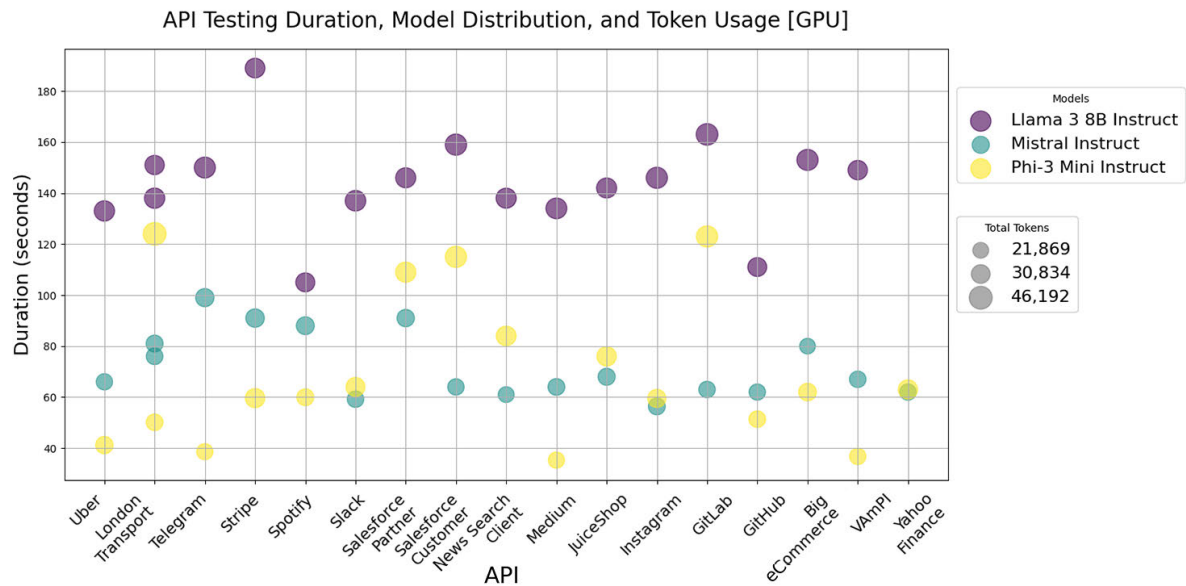


FIGURE 9. A visualisation of the session duration, total token usage (input+output) while using GPU as compute device.

performance improvements. Wizard v1.2 demonstrates significant resource utilization, requiring 24,486–41,619 tokens and processing durations ranging from 1 to 59.6 minutes, particularly for JuiceShop and Crapi APIs. This substantial resource commitment aligns with its significant performance improvements, suggesting that the model’s enhancement capabilities are resource-intensive but effective.

Llama 3 8B Instruct achieves an efficient balance, operating within a token range of 26,551–36,716 tokens and maintaining processing durations of approximately 2–3 minutes on GPU (extending to 44+ minutes on CPU). This efficiency is particularly noteworthy given its consistent performance improvements across APIs. The model’s resource utilization pattern indicates good optimization, achieving meaningful improvements with moderate computational overhead.

A clear efficiency frontier emerges in the resource utilization landscape:

- **High-Efficiency Zone:** Characterized by Llama 3 8B’s operations, showing optimal balance between resource consumption and performance gains;
- **Resource-Intensive Zone:** Dominated by Wizard v1.2, where substantial improvements come with significant computational costs.

The correlation between resource utilization and performance improvements reveals crucial insights for practical deployment considerations. Models demonstrating higher token efficiency (tokens processed per minute) generally show more consistent improvement patterns across metrics. This relationship suggests that efficient resource utilization might contribute to more stable and predictable performance enhancements.

The combined analysis indicates that optimal model selection should consider both the potential for performance

improvement and resource efficiency requirements. While Wizard v1.2 achieves the most substantial improvements in specific scenarios, its resource demands may limit practical applicability. Conversely, Llama 3 8B’s balanced profile of moderate improvements and efficient resource utilization presents a more versatile solution for diverse testing environments.

These findings have significant implications for automated testing frameworks, suggesting that resource efficiency should be considered alongside raw performance metrics when evaluating model suitability for specific testing scenarios. The observed patterns also indicate that future developments in test automation should focus on optimizing the relationship between resource utilization and performance improvements, potentially through more efficient training and refinement processes.

α: DISTRIBUTION OF ISSUES IN TEST CASES

The analysis of security test cases revealed a complex distribution of implementation challenges despite strong security testing intentions. Our data demonstrates robust security coverage, with 83.3% of test cases showing effective authentication and authorization mechanisms. However, the analysis also revealed notable technical implementation deficiencies, as seen in Fig 10.

| Issue Type              | Percentage |
|-------------------------|------------|
| Syntax Issues           | 44.4%      |
| Structural Issues       | 16.7%      |
| Authentication Coverage | 83.3%      |
| Authorization Coverage  | 83.3%      |

FIGURE 10. Distribution of Issues in Test Cases.

The technical challenges primarily manifested in syntax issues (44.4%), while structural deficiencies affected a smaller portion of the tests (16.7%). Despite these implementation challenges, the underlying security testing strategy demonstrated strong authentication and authorization coverage (83.3% for both), indicating robust security testing foundations.

Implementation issues typically centered around three main areas:

- Karate DSL syntax adherence, particularly in status assertions and request formatting;
- Test structure organization, especially in scenario outline formatting;
- Response validation implementation, notably in error handling and response matching.

This distribution suggests that while the fundamental security testing capabilities are robust, focusing remediation efforts on technical implementation aspects, particularly syntax adherence and structural organization, would significantly enhance test suite reliability and maintainability. The high coverage in authentication and authorization testing demonstrates that the framework successfully prioritizes security-critical aspects, even when facing technical implementation challenges.

#### E. EFFECTIVENESS OF RAG ENHANCEMENT

The consistent improvement in test quality and relevance across all models when using our RAG-enhanced approach underscores the value of contextual information in API security testing. The high interoperability scores (4.61-4.75) suggest that our method significantly reduces the need for manual intervention, a crucial factor for practical adoption in real-world testing scenarios.

#### F. BOLA-ORIENTED TEST GENERATION:

In our current tests, we focused exclusively on generating test scenarios designed to identify potential BOLA vulnerabilities. Our framework demonstrated 100% accuracy in generating BOLA-oriented test scenarios.

It's important to note that this perfect alignment indicates the framework's ability to generate tests specifically targeted at BOLA vulnerabilities when instructed to do so. However, this does not represent the detection of actual BOLA vulnerabilities in a target API, nor does it reflect the framework's performance in generating or identifying non-BOLA scenarios. Further testing with a more diverse set of intended scenario types would be necessary to fully evaluate the framework's discriminative capabilities.

##### 1) BOLA VULNERABILITY DETECTION

While specific detection rates were not provided, the improvement in accuracy and complexity scores suggests enhanced capability in identifying BOLA vulnerabilities compared to standard LLM approaches. However, the effectiveness may vary across different types of BOLA scenarios, warranting further investigation.

#### G. LIMITATIONS AND CHALLENGES

Despite the overall positive results of our experiments, we identified several significant limitations and challenges that warrant discussion. While our framework demonstrated promising test generation capabilities with BOLA oriented vulnerability checks, certain constraints and areas for improvement emerged:

- **Model Performance and Resources:** The performance gap between models suggests that larger, more resource-intensive models may be necessary for optimal results. Our specific configuration choices (context window size of 1000 tokens, temperature of 0.3) and hardware variations between CPU and GPU execution could significantly impact deployment in resource-constrained environments and limit real-time applications.
- **Vulnerability Coverage:** While our approach improved BOLA detection rates, it may not capture all possible vulnerability scenarios, particularly those requiring complex, multi-step interactions. This limitation highlights the need for more sophisticated test generation strategies. Additionally, our framework's primary focus on BOLA vulnerabilities might limit its effectiveness in detecting other types of API security vulnerabilities.
- **Model Dependencies:** The variability in performance across different LLMs indicates a dependency on the specific model used, which may introduce inconsistencies in test generation and affect the framework's reliability across different implementations.
- **API Selection Bias:** While we evaluated both intentionally vulnerable APIs (VAmPI, CrAPI, OWASP Juice Shop) and production APIs from APIRank, our results may not generalize to all API types, particularly those with unique authentication mechanisms or complex business logic. Although effective on our test set, the framework's performance on diverse real-world APIs with varying complexity requires further validation.
- **False Positives:** The potential for generating tests that incorrectly identify non-existent vulnerabilities needs further investigation and mitigation strategies to ensure the accuracy and reliability of the generated test cases.
- **Test Quality Assessment:** Our evaluation metrics (Accuracy, Interoperability, Complexity) might not capture all relevant aspects of test case quality and effectiveness.

##### 1) GENERAL CODE QUALITY METRICS NOT INCLUDED

While the following aspects are important for overall code quality assessment, they have been deliberately excluded from our primary evaluation framework:

##### 1) Code Maintainability

- Readability and documentation quality;
- Code structure and organization;
- Comment completeness and clarity.

These aspects, while valuable, are general software engineering concerns that apply to all code rather than being specific to security test generation.

## 2) Cognitive Complexity

- Intellectual effort required to understand the test;
- Learning curve for new team members;
- Mental model complexity.

These factors are inherently subjective and may vary based on individual expertise and background.

## 3) Dependencies and Environmental Factors

- External library dependencies;
- Environment setup requirements;
- Cross-platform compatibility.

These considerations are more relevant to long-term maintenance than to the immediate quality of generated security tests.

## H. RATIONALE FOR EXCLUSION

Our decision to exclude these general code quality metrics is based on several key considerations:

- 1) **Focus on Generation:** Our metrics prioritize aspects directly related to the quality of automatically generated security tests rather than general code quality concerns.
- 2) **Objectivity:** The chosen metrics emphasize objectively measurable aspects of test quality specific to BOLA vulnerability detection.
- 3) **Automation Compatibility:** The selected metrics can be evaluated programmatically, making them suitable for automated assessment of generated tests.
- 4) **Domain Specificity:** The framework focuses on security testing characteristics rather than general software engineering principles.

Despite these limitations, we have implemented several strategies to strengthen the validity of our findings:

- **Standardized Evaluation Environment:** We maintained consistent configurations across all experiments, including standardized token windows (1000 tokens) and temperature settings (0.3), allowing for fair comparisons between models.
- **Diverse API Testing:** To address generalizability concerns, we expanded our evaluation beyond vulnerable test APIs to include production APIs spanning various domains, including social media platforms, version control systems, and financial services.
- **Comprehensive Metrics:** Our evaluation framework employs multiple complementary metrics (Accuracy, Interoperability, Complexity) to provide a holistic assessment of test case quality, helping offset potential limitations in individual metrics.
- **Reproducibility Focus:** We have made our implementation, including the complete dataset of generated tests and analysis results, publicly available in our project repository for transparency and reproducibility.

- **Model Performance Documentation:** We thoroughly documented performance characteristics across both CPU and GPU configurations, providing clear guidance for deployment decisions in different resource environments.

While these strategies help mitigate our identified limitations, we acknowledge that further research is needed to address remaining challenges in automated API security testing.

## I. IMPLICATIONS FOR API SECURITY TESTING

Our experimental results with the VAmPI API, OWASP crAPI, and OWASP Juice Shop revealed significant insights into the performance characteristics of different LLMs within our RAG-enhanced framework. The following analysis focuses on processing time aspects across test generation and execution:

### • Initial Generation Times:

- Mistral 7B demonstrated efficient initial generation (24.52s average);
- Phi-3 Mini showed comparable performance (26.99s average);
- Llama 3 8B required moderate time (28.01s average);
- Wizard v1.2 showed significantly longer generation times (691.0s average).

### • Verification Phase:

- Phi-3 Mini exhibited the fastest verification (22.91s);
- Llama 3 8B averaged 37.05s for verification;
- Mistral 7B required 59.96s;
- Wizard v1.2 needed substantially more time (418.0s).

### • Total Processing Overview:

- Phi-3 Mini and Mistral 7B showed the most consistent performance across all stages;
- Llama 3 8B demonstrated balanced performance with moderate processing times;
- Wizard v1.2 consistently required significantly more processing time across all stages.

These timing results have important implications for practical deployment:

- 1) **Resource Allocation:** The significant variations in processing times between models (e.g., 17.16s vs 217.0s for test execution) suggest the need for careful resource planning based on specific testing requirements.
- 2) **Pipeline Optimization:** The consistent performance of Phi-3 Mini and Mistral 7B across all stages indicates their suitability for continuous integration environments where processing time is critical.
- 3) **Scalability Considerations:** The processing time differences become particularly significant when considering large-scale API testing scenarios, suggesting

the potential need for model-specific optimization strategies.

These findings contribute to our understanding of practical LLM deployment in security testing frameworks and provide concrete guidance for model selection based on timing constraints and resource availability.

Our findings have several implications for the field of API security testing:

- **Automation Potential:** The high accuracy and interoperability scores demonstrate the potential for significant automation in API security testing, potentially reducing the reliance on manual expertise;
- **Contextual Importance:** The effectiveness of our RAG approach highlights the critical role of contextual information in generating relevant and effective security tests;
- **Model-Specific Strategies:** The varied performance across LLMs suggests the need for tailored strategies depending on the specific requirements of the testing scenario (e.g., speed vs. depth);
- **Iterative Improvement:** Our framework's ability to learn and improve through iterations points to the potential for continuously evolving test suites that adapt to emerging vulnerabilities.

These findings provide valuable insights into the effectiveness of our RAG-enhanced approach and highlight areas for future improvement in API security testing methodologies.

## V. CONCLUSION

This paper introduced Karate-BOLA-Guard, a novel framework for automated API security testing that leverages LLMs and RAG techniques to detect BOLA vulnerabilities at the API Level.

Our research makes several significant contributions to the field of API security testing:

- A modular, self-improving framework integrating LLMs, RAG, and iterative refinement for enhanced API security testing;
- A novel approach to generating context-aware, security-focused test cases using Karate DSL;
- Empirical evidence demonstrating the effectiveness of RAG techniques in improving BOLA vulnerability detection.

### A. ADDRESSING RESEARCH QUESTIONS

Our comprehensive experimentation with Karate-BOLA-Guard yielded significant insights into the effectiveness and implications of RAG-enhanced API security testing. Through rigorous analysis, we present our findings addressing each research question, revealing complex patterns and relationships in automated security test generation.

#### 1) RQ 1: RAG ENHANCEMENT OF TEST CASE QUALITY AND RELEVANCE

The integration of RAG techniques demonstrated measurable improvements in both quality and relevance of generated test cases. Our analysis revealed that RAG enhancement significantly impacted all three key metrics defined in our evaluation framework: Accuracy, Interoperability, and Complexity. As evidenced in Table 4, Llama 3 8B consistently achieved the highest performance across these metrics, particularly in handling complex API architectures and authentication scenarios.

The enhancement in test quality was particularly evident in more sophisticated APIs, where we observed higher Accuracy and Interoperability scores, notably in platforms like Instagram (A: 4.3, I: 4.4) and GitLab (A: 4.2, I: 4.2). This pattern suggests that RAG enhancement was particularly effective in generating contextually aware tests for complex authentication and authorization scenarios. Furthermore, as shown in Figure 6, the iterative refinement process consistently improved test quality while maintaining strong security coverage, especially in authentication and authorization testing scenarios.

#### 2) RQ 2: IMPACT ON COVERAGE AND SPECIFICITY

Specifically, regarding BOLA vulnerability testing, our analysis shows that test specificity varied significantly across different APIs, with Llama 3 8B demonstrating notable performance patterns across a broader range of APIs:

- **Test Coverage (Accuracy):** While the initial analysis showed accuracy ranges of 2.7-3.4, our expanded evaluation reveals a wider range (2.6-4.3), with particularly strong performance in modern API platforms:
  - High performance in commercial APIs (Instagram: 4.3, GitLab: 4.2, JuiceShop: 4.2);
  - Lower performance in simpler APIs (Crapi: 2.6, Yahoo: 3.0).
- **Test Sophistication (Complexity):** Complexity scores show greater variation (2.4-4.4) than initially observed, indicating more nuanced test generation capabilities:
  - Advanced scenarios in e-commerce and social platforms (Big-Commerce: 4.4, Instagram: 4.4);
  - More sophisticated test structures for OAuth-based APIs (GitLab: 4.2, JuiceShop: 4.2).
- **Framework Integration (Interoperability):** The extended analysis confirms a wide variation in interoperability (2.5-4.4), with:
  - Strong performance in complex authentication scenarios (Instagram: 4.4, JuiceShop: 4.3, VAmPI: 4.3);
  - Lower scores in APIs with simpler authentication mechanisms (Crapi: 2.5, Yahoo: 2.6).

These expanded results demonstrate that the framework's effectiveness in generating BOLA-specific test cases correlates strongly with API complexity and authentication mech-



anisms, particularly excelling in APIs with sophisticated security requirements.

### 3) RQ 3: TRADE-OFFS IN RAG-ENHANCED TESTING

Our investigation of trade-offs in RAG-enhanced testing revealed significant patterns in performance metrics, resource utilization, and processing times. The data presented in Table 6 and demonstrates several key trade-offs:

#### 1) Processing Time vs. Accuracy:

- Llama 3 8B achieved the highest accuracy scores (3.1-4.3) but required longer processing times (average 143.76s);
- Smaller models like Phi-3 Mini offered faster processing (69.58s average) but showed lower accuracy (2.8-3.3);
- The performance differential between CPU and GPU execution was substantial (20-25x improvement with GPU).

#### 2) Resource Consumption vs. Test Quality:

- Larger models demonstrated better test quality but required significantly more computational resources;
- Llama 3 8B used the most tokens (36,591 average) but produced the most comprehensive tests;
- Mistral showed efficient resource usage (25,225 tokens) while maintaining acceptable quality metrics.

#### 3) Model Size vs. Consistency:

- Larger models showed more consistent performance across different APIs (Llama 3 8B CoV: 13.20%);
- Smaller models exhibited higher variability (Phi-3 Mini CoV: 42.45%);
- This consistency came at the cost of increased processing time and resource requirements.

These findings suggest a clear correlation between resource investment and test quality, with higher resource allocation generally yielding better results. However, the relationship is not strictly linear - Llama 3 8B represents an optimal balance point, offering strong performance improvements without the extreme resource demands of larger models like Wizard v1.2. As shown in Figure 5, these trade-offs manifest differently across various API types and testing scenarios. The results indicate that organizations must carefully consider their specific requirements and resource constraints when implementing RAG-enhanced testing solutions. The choice between faster processing times and higher accuracy depends largely on the specific use case and available computational resources. These insights contribute valuable insight to the practical implementation of RAG-enhanced API security testing, highlighting the importance of balanced resource allocation and performance optimization in real-world deployments.

## VI. FUTURE RESEARCH

Our findings point to several promising avenues for future research in automated API security testing. The development of dynamic model selection mechanisms presents an immediate opportunity for advancement, potentially incorporating real-time API characteristic analysis for optimal model deployment. Resource optimization remains a critical area for investigation, particularly in the context of large-scale API testing environments.

The potential for cross-model learning mechanisms presents an intriguing direction for future research, which could enable the development of hybrid approaches that combine the strengths of different model architectures. Advanced RAG techniques, particularly in the realm of context-aware embedding generation and adaptive similarity metrics, offer promising opportunities for improving test generation accuracy and efficiency.

Furthermore, the development of sophisticated performance prediction models could significantly enhance the practical utility of the framework. Such models could incorporate API complexity assessment and resource utilization forecasting, enabling more efficient resource allocation in large-scale testing environments. These research directions, grounded in our empirical findings, suggest a rich landscape for future investigation in the field of automated API security testing.

In conclusion, Karate-BOLA-Guard represents a significant step forward in automated API security testing, particularly for BOLA vulnerabilities. Using the power of LLM and RAG techniques, our framework offers a more efficient, accurate, and adaptable approach to identifying and mitigating critical API security risks. As APIs continue to play a central role in modern software architectures, tools like Karate-BOLA-Guard will be crucial in ensuring the security and integrity of these vital components.

## VII. IMPLEMENTATION AND AVAILABILITY

The complete implementation of our framework is available as open-source software on GitHub.<sup>11</sup> The repository includes all components described in this paper, along with documentation and example configurations.

## REFERENCES

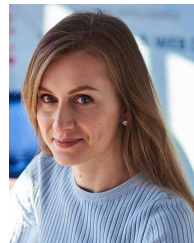
- [1] OWASP API Security Project Team. (Dec. 2023). *API:2023 Broken Object Level Authorization—OWASP API Security Top 10*. Accessed: Jan. 24, 2024. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/Oxa1-broken-object-level-authorization>
- [2] E. M. Pasca, R. Erdei, D. Delinschi, and O. Matei, "Overview of machine learning processes used in improving security in API-based web applications," in *Proc. Comput. Sci. On-Line Conf.* Springer, 2023, pp. 367–381, doi: 10.1007/978-3-031-35314-7\_33.
- [3] G. Baye, F. Hussain, A. Oracevic, R. Hussain, and S. M. A. Kazmi, "API security in large enterprises: Leveraging machine learning for anomaly detection," in *Proc. Int. Symp. Netw., Comput. Commun. (ISNCC)*, Oct. 2021, pp. 1–6.

<sup>11</sup><https://github.com/EmilMarian/Karate-BOLA-Guard-Framework>

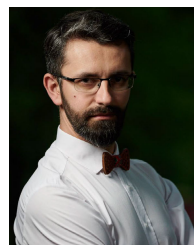
- [4] M. Sowmya, A. J. Rai, V. Spoorthi, M. Irfan, P. B. Honnavalli, and S. Nagasundari, "API traffic anomaly detection in microservice architecture," in *Proc. IEEE/ACM 23rd Int. Symp. Cluster, Cloud Internet Comput. Workshops (CCGridW)*, May 2023, pp. 206–213.
- [5] A. Aumpansub and Z. Huang, "Detecting software vulnerabilities using neural networks," in *Proc. 13th Int. Conf. Mach. Learn. Comput.* New York, NY, USA: Association for Computing Machinery, Feb. 2021, pp. 166–171, doi: [10.1145/3457682.3457707](https://doi.org/10.1145/3457682.3457707).
- [6] A. Barabanov, D. Dergunov, D. Makrushin, and A. Teplov, "Automatic detection of access control vulnerabilities via API specification processing," *Voprosy Kiberbezopasnosti*, vol. 1, pp. 49–65, Jan. 2022, doi: [10.21681/2311-3456-2022-1-49-65](https://doi.org/10.21681/2311-3456-2022-1-49-65).
- [7] Y. Huang, C. Shi, J. Lu, H. Li, H. Meng, and L. Li, "Detecting broken object-level authorization vulnerabilities in database-backed applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.* New York, NY, USA: Association for Computing Machinery, Dec. 2024, pp. 2934–2948, doi: [10.1145/3658644.3690227](https://doi.org/10.1145/3658644.3690227).
- [8] A. G. Mirabella, A. Martin-Lopez, S. Segura, L. Valencia-Cabrera, and A. Ruiz-Cortés, "Deep learning-based prediction of test input validity for RESTful APIs," in *Proc. IEEE/ACM 3rd Int. Workshop Deep Learn. Test. Test. Deep Learn. (DeepTest)*, Jun. 2021, pp. 9–16. [Online]. Available: <https://api.semanticscholar.org/CorpusID>
- [9] M. Bhatt et al., "Purple llama CyberSecEval: A secure coding benchmark for language models," 2023, *arXiv:2312.04724*.
- [10] M. L. Siddiq, J. C. D. S. Santos, S. Devareddy, and A. Müller, "SALLM: Security assessment of generated code," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng. Workshops*, Oct. 2024, pp. 54–65, doi: [10.1145/3691621.3694934](https://doi.org/10.1145/3691621.3694934).
- [11] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, "RESTest: Automated black-box testing of RESTful web APIs," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2021, pp. 682–685.
- [12] H. Ed-douibi, J. L. Izquierdo, and J. Cabot, "Automatic generation of test cases for REST APIs: A specification-based approach," in *Proc. IEEE 22nd Int. Enterprise Distrib. Object Comput. Conf. (EDOC)*, Oct. 2018, pp. 181–190.
- [13] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024.
- [14] M. Kim, T. Stennett, S. Sinha, and A. Orso, "A multi-agent approach for REST API testing with semantic graphs and LLM-driven inputs," 2024, *arXiv:2411.07098*.
- [15] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, Apr. 2024.
- [16] D. M. Zapkus and A. Slotkien. (2024). *Unit Test Generation Using Large Language Models: A Systematic Literature Review*. Vilnius Univ. Open Ser. [Online]. Available: <https://api.semanticscholar.org/CorpusID:269810554>
- [17] L. Huang, P. Zhao, H. Chen, and L. Ma, "Large language models based fuzzing techniques: A survey," 2024, *arXiv:2402.00350*.
- [18] K. Shashwat, F. Hahn, X. Ou, D. Goldgof, L. Hall, J. Ligatti, S. R. Rajgopalan, and A. Z. Tabari, "A preliminary study on using large language models in software pentesting," 2024, *arXiv:2401.17459*.
- [19] R. Kande, H. Pearce, B. Tan, B. Dolan-Gavitt, S. Thakur, R. Karri, and J. Rajendran, "(Security) assertions by large language models," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4374–4389, 2024.
- [20] R. Santos, I. Santos, C. Magalhaes, and R. D. S. Santos, "Are we testing or being tested? Exploring the practical applications of large language models in software testing," 2023, *arXiv:2312.04860*.
- [21] (Aug. 2024). *Introduction to LangChain*. Accessed: Aug. 27, 2024. [Online]. Available: <https://python.langchain.com/v0.2/docs/introduction>
- [22] A. M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, "Effective test generation using pre-trained large language models and mutation testing," 2023, *arXiv:2308.16557*.
- [23] C. Tony, M. Mutas, N. E. D. Ferreyra, and R. Scandariato, "LLMSEval: A dataset of natural language prompts for security evaluations," 2023, *arXiv:2303.09384*.
- [24] E. M. Paşca, R. Erdei, D. Delinschi, and O. Matei, "Augmenting API security testing with automated LLM-driven test generation," in *Proc. Comput. Intell. Secur. Inf. Syst. Conf.* Springer, Jan. 2024, pp. 112–121, doi: [10.1007/978-3-031-75016-8\\_11](https://doi.org/10.1007/978-3-031-75016-8_11).
- [25] (Feb. 2025). *API RANK | Benchmarks of Public APIs*, by Escape. Accessed: Feb. 2, 2025. [Online]. Available: <https://apirank.dev>
- [26] E. M. Pasca. (2024). *Bola Dataset for Karate LLM Project*. [Online]. Available: <https://www.kaggle.com/dsv/7869548>
- [27] E. M. Paşca, R. Erdei, D. Delinschi, and O. Matei, "Enhancing API security testing against BOLA and authentication vulnerabilities through an LLM-enhanced framework," in *Proc. Int. Workshop Soft Comput. Models Ind. Environ. Appl.* Springer, Nov. 2024, pp. 231–240, doi: [10.1007/978-3-031-75010-6\\_23](https://doi.org/10.1007/978-3-031-75010-6_23).



**EMIL MARIAN PASCA** is currently pursuing the Ph.D. degree with a focus on cybersecurity, distributed systems, and machine learning. His research interests include web applications, automation testing, secure software development, and machine learning, forming a niche at the intersection of cutting-edge technology and security.



**DANIELA DELINSCHI** is currently pursuing the Ph.D. degree in computer science (data processing and machine learning). She has a background in IoT and system optimization. She is working on numerous software projects with the Holisun Research and Development Team. Her expertise encompasses electronic system engineering, the IoT design and integrations, data mining, and cloud computing.



**RUDOLF ERDEI** received the Ph.D. degree in computer science (distributed systems and artificial intelligence). He has a background in software engineering and artificial intelligence. Currently, he is an Assistant Professor at the Technical University of Cluj-Napoca, North University Centre of Baia Mare in the field of Computer Engineering. His research interests include metaheuristic algorithms, distributed systems, cloud/fog/edge computing, and optimizing VRP class problems.



**OLAVIU MATEI** received the M.S. degree in artificial intelligence from Vrije Universiteit Amsterdam, in 2002, the Ph.D. degree in systems engineering from the Technical University of Cluj-Napoca, Romania, in 2012, and the Habilitation degree in computer science and information technology, in 2016. Since 2002, he has been with the Technical University of Cluj-Napoca, where he is currently a Full Professor. He supervised a group of Ph.D. students. His research interests

include artificial intelligence with evolutionary computation, evolutionary ontologies, optimization, and robotics.

...